

TESTING DEPLOYMENT PIPELINE AND

AUTOMATING PRODUCTION PREPARATION TASK

[NAME……]

IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF

BACHELOR OF ……

FACULTY OF …….

THAI-NICHI INSTITUTE OF TECHNOLOGY

[YEAR]

TESTING DEPLOYMENT PIPELINE AND

AUTOMATING PRODUCTION PREPARATION TASK

[NAME……..]

IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF

BACHELOR OF ……

FACULTY OF …….

THAI-NICHI INSTITUTE OF TECHNOLOGY

[YEAR]

Approval of the Committee

……. The Chairperson of Approving Committee

(…….)

……. Committee Members

(…….)

……. Faculty Advisor

(…….)

……. The Chairperson of Co-operative Education

(…….)

Copyright © by Thai-Nichi Institute of Technology

Project's name	TESTING DEPLOYMENT PIPELINE AND AUTOMATING PRODUCTION PREPARATION TASK
Writer	……
Faculty	Information Technology, Data Science and Analytics
Faculty Advisor	…….
Job Supervisor	…….
Company's name	…….
Business Type / Product	**…….**

Summary

……

……..…

……..

……

……..…

……..

Acknowledgement

……

……...…

……..

……

……...…

……..

……

……...…

……..

[NAME…….]

Table of Contents

Page

Summary i

Acknowledgement ii

Table of Contents iii

List of Figures vi

List of Tables ix

1 Introduction 1

1.1 Name and location of the company 1

1.2 Type of business or service of the company 2

1.3 Assigned position and duties that the student is responsible for 2

1.4 Job Supervisor from the company and position 2

1.5 The length of time for the assigned position 2

1.6 Sources and importance of occurred problems 2

1.7 Objectives/aims of the project 3

1.8 Expected result of the project or assignment 3

1.9 Definitions of jargon in the field 3

2 Theory and Technology Used 5

2.1 Theories used in practice 5

2.1.1 Software Development Life Cycle 5

2.1.2 SQL 5

2.1.3 Python 13

2.1.4 Linux command 14

2.2 Technologies used in practice 17

2.2.1 SourceTree 17

2.2.2 MobaXterm 18

2.2.3 Jenkins 19

2.2.4 Visual Studio Code 19

2.2.5 MS Excel 20

3 Work Plan and Procedures 22

3.1 Plans of the Project 22

3.2 Detailed Procedures of the Project 23

3.2.1 Project Background 23

3.2.2 Jenkins Deployment Pipeline 23

3.2.3 Production Preparation 24

3.3 Practical Procedures Used for the Project 24

3.3.1 Practical Procedures of Jenkins Deployment Pipeline 24

3.3.2 Practical Procedures of Production Preparation 28

4 Result of the Project 51

4.1 Result of the Work 51

4.1.1 Result of Jenkins Deployment Pipeline 51

4.1.2 Result of Production Preparation 51

4.2 Data Analysis 52

4.2.1 Data Analysis of Jenkins Deployment Pipeline 52

4.2.2 Data Analysis of Production Preparation 53

4.3 Analysis of Results Comparing to Objectives and Aims of the Assignment 53

5 Conclusion and Recommendations 54

5.1 Conclusions 54

5.2 Problems and Possible Solutions 54

References 55

Appendix 57

Sign Off 80

Background of the Writer 81

List of Figures

Pages

Figure 1.1 Company's Logo 1

Figure 1.2 Company's Map 1

Figure 2.1 Output of GROUP BY clause. 11

Figure 2.2 Output of PARTITION BY clause. 12

Figure 2.1 Example of pwd command 15

Figure 2.2 Example of mkdir command 15

Figure 2.3 Example of rmdir command 16

Figure 2.4 Example of ls command 16

Figure 2.5 Example of cd command 16

Figure 2.6 Example of cat command 17

Figure 2.7 Screenshot of SSMS user interface. 18

Figure 2.8 Screenshot of MobaXterm window screen. 18

Figure 2.9 Screenshot of Jenkins user interface 19

Figure 2.10 Screenshot of Visual Studio Code interface. 20

Figure 2.11 Screenshot of Excel interface. 21

Table 3.1 Plans of the project 22

Figure 3.1 Preparing Input Excel Files. 25

Figure 3.2 Commit files used in testing to GIT. 25

Figure 3.3 Run pipeline on Jenkins. 26

Figure 3.4 Jenkins generate log on console output. 26

Figure 3.5 Jenkins test excel, the worksheet shows test scenarios, test result status, link to result screen capturing and input and output file name. 27

Figure 3.6 Jenkins test excel, the worksheet shows the screen capture of Jenkins console output on each scenario. 27

Figure 3.7 Jenkins Deployment Transition Presentation slides. 28

Figure 3.8 Workflow chart of validating file with inventory and compare last modified date. 29

Figure 3.9 Screenshot of sample script inventory excel input file. 29

Figure 3.10 Screenshot of validate file with inventory and compare last modified date of file code 30

Figure 3.11 Screenshot of validate file with inventory and compare last modified date of file code (cont.) 31

Figure 3.12 Screenshot of validate file with inventory and compare last modified date of file output. 31

Figure 3.13 Screenshot of validate file with inventory and compare last modified date of file output (cont.) 32

Figure 3.14 Workflow chart of validating table and view inventory. 33

Figure 3.15 Screenshot of table and view inventory input excel file. 33

Figure 3.16 Screenshot of validating table and view inventory code. 34

Figure 3.17 Screenshot of validating table and view inventory code (cont.) 35

Figure 3.18 Screenshot of validating table and view inventory output 35

Figure 3.19 Screenshot of validating table and view inventory output (cont.) 36

Figure 3.20 Workflow of finding as is distinct IP Address. 36

Figure 3.21 Workflow of replacing IP address. 37

Figure 3.22 Screenshot of inut csv file. 37

Figure 3.23 Screenshot of getting a distinct IP address code. 38

Figure 3.24 Screenshot of getting a distinct IP address output 38

Figure 3.25 Screenshot of generateing files with replaced IP address code. 39

Screenshot of generating files with replaced IP address output. 40

Figure 3.26 Workflow chart of validating date function. 41

Figure 3.27 Screenshot of input excel file. 41

Figure 3.28 Screenshot of automate test date function code. 42

Figure 3.29 Screenshot of automate test date function code (cont.) 43

Figure 3.30 Screenshot of automate test date function code (cont.) 44

Figure 3.31 Screenshot of automate test date function output. 45

Figure 3.32 Screenshot of auto test date function output excel file. 46

Figure 3.33 Workflow chart of validating insert overwrite with empty result. 46

Figure 3.34 Screenshot of validating insert overwrite with empty result code. 47

Figure 3.35 Screenshot of validating insert overwrite with empty result output. 48

Figure 3.36 Workflow chart of validating drop partition with varchar datatype. 48

Figure 3.37 Screenshot of validating drop partition with varchar datatype code. 49

Figure 3.38 Screenshot of validating drop partition with varchar datatype output. 49

Figure 3.39 Transition document preparation. 50

List of Tables

Pages

Table 3.1 Plans of the project 22

Chapter 1

Introduction

1.1 Name and location of the company

Name of the company xxxxxxxxxxxxxxxxxxxxxxx

Location xxxxxxxxxxxxxxxxxxxxxxx

Figure 1.1 Company's Logo

Figure 1.2 Company's Map

1.2 Type of business or service of the company

Accenture Solutions is a services company, providing a broad range of services and solutions in strategy, consulting, digital, technology, and operations.

1.3 Assigned position and duties that the student is responsible for

Position xxxxxxxxxxxxxxxxxxxx

Job description

1. xxxxxxxxxxxxxxxxxxxx
2. xxxxxxxxxxxxxxxxxxxx
3. xxxxxxxxxxxxxxxxxxxx

1.4 Job Supervisor from the company and position

Job Supervisor xxxxxxxxxxxxxxxxxxxx

Position xxxxxxxxxxxxxxxxxxxx

1.5 The length of time for the assigned position

xxxxxxxxxxxxxxxxxxxx

1.6 Sources and importance of occurred problems

……

……..…

……..

……

……..…

……..

1.7 Objectives/aims of the project

1. xxxxxxxxxxxxxxxxxxxxx
2. xxxxxxxxxxxxxxxxxxxxx

1.8 Expected result of the project or assignment

3. xxxxxxxxxxxxxxxxxxxxx
4. xxxxxxxxxxxxxxxxxxxxx

1.9 Definitions of jargon in the field

Data Lake - a central storage repository that holds big data from many sources in a raw, granular format. It can store structured, semi-structured, or unstructured data, which means data can be kept in a more flexible format for future use. Data lakes are easily confused with data warehouses but feature some distinct differences that can offer big benefits to the right organizations—especially as big data and big data processes continue to migrate from on-premises to the cloud.

PaaS - Cloud platform services, also known as Platform as a Service (PaaS), provide cloud components to certain software while being used mainly for applications. PaaS delivers a framework for developers that they can build upon and use to create customized applications. All servers, storage, and networking can be managed by the enterprise or a third-party provider while the developers can maintain management of the applications.

Azure HDInsight - Azure HDInsight or HDI is a cloud distribution of Hadoop components. Azure HDInsight makes it easy, fast, and cost-effective to process massive amounts of data. The user can utilize many popular open-source frameworks such as Hadoop, Spark, Hive, LLAP, Kafka, Storm, R, and more. These frameworks can enable a broad range of scenarios such as extract, transform, and load (ETL), data warehousing, machine learning, and IoT.

ETL a type of data integration that refers to the three steps (extract, transform, load) used to blend data from multiple sources. It's often used to build a data warehouse. During this process, data is taken (extracted) from a source system, converted (transformed) into a format that can be analyzed, and stored

(loaded) into a data warehouse or other system. Extract, load, transform (ELT) is an alternate but related approach designed to push processing down to the database for improved performance.

Framework - is a platform for developing software applications. It provides a foundation on which software developers can build programs for a specific platform.

Automation - the technology by which a process or task is performed with no assistance from a person. Automation can refer to machines like robots or programs that carry out repetitive tasks in software applications.

Chapter 2

Theory and Technology Used

2.1 Theories used in practice

2.1.1 Software Development Life Cycle

Software Development Life Cycle (SDLC) is a process that produces software with the highest quality and lowest cost in the shortest time possible. SDLC provides a well-structured flow of phases that help an organization to quickly produce high-quality software which is well-tested and ready for production use.

The Software Development Life Cycle (SDLC) refers to a methodology with clearly defined processes for creating high-quality software. The SDLC methodology focuses on the following phases of software development:

1. Requirement analysis - identify the current problems with improvement as a goal
2. Plan – determines the cost and resources for implementing the analyze requirements
3. Design – making a design plan from the software specifications
4. Build or Develop – developing the software
5. Testing – test to verify that the code meets the defined requirements
6. Deployment – deploy software to production environment for a go or no-go decision

2.1.2 SQL

SQL is Structured Query Language, which is a computer language for storing, manipulating, and retrieving data stored in a relational database. It is the standard language for Relational Database System. All the Relational Database Management Systems (RDMS) use SQL as their standard database language.

SQL Commands

The standard SQL commands to interact with relational databases are CREATE, SELECT, INSERT, UPDATE, DELETE and DROP. These commands can be classified into the following groups based on their nature —

DDL - Data Definition Language

1. CREATE - Creates a new table, a view of a table, or other object in the database.
2. ALTER - Modifies an existing database object, such as a table.
3. DROP - Deletes an entire table, a view of a table or other objects in the database.

DML - Data Manipulation Language

1. INSERT - Creates a record.
2. SELECT - Retrieves certain records from one or more tables.
3. UPDATE - Modifies records.
4. DELETE - Deletes records.

DCL - Data Control Language

1. GRANT - Gives a privilege to user.
2. REVOKE - Takes back privileges granted from user.

SQL Command Usage

The following will explain the usage for each command and useful clauses that will make SQL more practical. However, the topic will exclude DCL set command since it is not used under the making of this report.

CREATE Command

1. CREATE DATABASE statement is used to create a new SQL database.

   Syntax

   ```
   CREATE DATABASE DatabaseName;
   ```

   Example

   ```
   CREATE DATABASE testDb;
   ```

2. CREATE TABLE statement is used to create a new table.

   Syntax

   ```
   CREATE TABLE table_name(
      column1 datatype,
   ```

```
   column2 datatype,
   column3 datatype,
   .....
   columnN datatype,
   PRIMARY KEY( one or more columns )
);
```

Example

```
CREATE TABLE CUSTOMERS(
   ID   INT              NOT NULL,
   NAME VARCHAR (20)     NOT NULL,
   AGE  INT              NOT NULL,
   ADDRESS  CHAR (25) ,
   SALARY   DECIMAL (18, 2),
   PRIMARY KEY (ID)
);
```

ALTER TABLE Command

ALTER TABLE statement is used to add, delete, or modify columns in an existing table. It is also use when adding and dropping various constraints on an existing table.

Syntax

```
ALTER TABLE table_name ADD column_name datatype;
```

```
ALTER TABLE table_name DROP COLUMN column_name;
```

```
ALTER TABLE table_name MODIFY COLUMN column_name datatype;
```

Example

```
ALTER TABLE CUSTOMERS ADD SEX char(1);
```

```
ALTER TABLE CUSTOMERS DROP SEX;
```

DROP Command

1 DROP DATABASE statement is used to drop an existing database in SQL schema.

Syntax

```
DROP DATABASE DatabaseName;
```

Example

```
DROP DATABASE testDb;
```

2 DROP TABLE statement is used to remove a table definition and all the data, indexes, triggers, constraints and permission specifications for that table.

Syntax

```
DROP TABLE Table_Name;
```

Example

```
DROP TABLE CUSTOMERS;
```

INSERT Command

INSERT INTO Statement is used to add new rows of data to a table in the database.

Syntax

```
INSERT INTO TABLE_NAME (column1,column2,...columnN)
VALUES (value1,value2,value3,...valueN);
```

```
INSERT INTO TABLE_NAME VALUES (value1,value2,...valueN);
```

Example

```
INSERT INTO CUSTOMERS (ID,NAME,AGE,ADDRESS,SALARY)
VALUES (1, 'Ramesh', 32, 'Ahmedabad', 2000.00 );
```

```
INSERT INTO CUSTOMERS
VALUES (2, 'Muffy', 24, 'Indore', 10000.00 );
```

SELECT Command

SELECT statement is used to fetch the data from a database table which returns this data in the form of a result table.

Syntax

```
SELECT column1, column2, columnN FROM table_name;
```

Example

Fetch all the fields available in the field.

```
SELECT * FROM table_name;
```

Fetch a specific field available in CUSTOMERS table.

```
SELECT ID, NAME, SALARY FROM CUSTOMERS;
```

WHERE clause is used to specify a condition while fetching the data from a single table or by joining with multiple tables. If the given condition is satisfied, then only it returns a specific value from the table. The clause is not only used in the SELECT statement, but it is also used in the UPDATE, DELETE statement, etc. It can specify a condition using the comparison or logical operators like >, <, =, LIKE, NOT, etc.

Syntax

```
SELECT column1, column2, columnN
FROM table_name
WHERE [condition]
```

Example

Fetch the ID, Name and Salary fields from the CUSTOMERS table, where the salary is greater than 2000.

```
SELECT ID, NAME, SALARY
FROM CUSTOMERS
WHERE SALARY > 2000;
```

Fetch the ID, Name and Salary fields from the CUSTOMERS table for a customer with the name Hardik.

```
SELECT ID, NAME, SALARY
FROM CUSTOMERS
WHERE NAME = 'Hardik';
```

AND & OR clauses (operators) are used to combine multiple conditions to narrow data in an SQL statement. The records will be shown following these consequences:

- all conditions separated by the AND must be TRUE
- the only any ONE of the conditions separated by the OR must be TRUE.

Syntax

AND clause

```
SELECT column1, column2, columnN
FROM table_name
```

```
WHERE [condition1] AND [condition2]...AND [conditionN];
```

OR clause

```
SELECT column1, column2, columnN
FROM table_name
WHERE [condition1] OR [condition2]...OR [conditionN]
```

Example

Fetch the ID, Name and Salary fields from the CUSTOMERS table, where the salary is greater than 2000 **AND** the age is less than 25 years.

```
SELECT ID, NAME, SALARY
FROM CUSTOMERS
WHERE SALARY > 2000 AND age < 25;
```

Fetch the ID, Name and Salary fields from the CUSTOMERS table, where the salary is greater than 2000 **OR** the age is less than 25 years.

```
SELECT ID, NAME, SALARY
FROM CUSTOMERS
WHERE SALARY > 2000 OR age < 25;
```

GROUP BY clause is used in collaboration with the SELECT statement to arrange identical data into groups. This GROUP BY clause follows the WHERE clause in a SELECT statement. The clause must follow the conditions in the WHERE clause and must precede the ORDER BY clause if one is used.

Aggregate functions such as MAX(), MIN(), AVERAGE() and COUNT() is used with GROUP BY clause to perform data analysis. It reduces the number of rows returned by rolling them up and calculating the aggregate function for each group.

Syntax

```
SELECT column1, column2
FROM table_name
WHERE [ conditions ]
GROUP BY column1, column2
ORDER BY column1, column2
```

Example

Getting the total amount of the salary on each customer

```
SELECT department_id,
       ROUND(AVG(salary))avg_department_salary
FROM employees
GROUP BY department_id
ORDER BY department_id;
```

department_id	avg_department_salary
1	4400
2	9500
3	4150
4	6500
5	5886
6	5760
7	10000
8	9617
9	19333
10	8600
11	10150

Figure 2.1 Output of GROUP BY clause.

PARTITION BY clause is a subclause of the OVER clause. The PARTITION BY clause divides a query's result set into partitions.

Syntax

```
window_function ( expression ) OVER (
   PARTITION BY expression1, expression2, ...
   order_clause
   frame_clause
);
```

Example

Getting the total amount of the salary on each customer

```
SELECT first_name, last_name, department_id,
       ROUND(AVG(salary) OVER(
             PARTITION BY department_id)
             )avg_department_salary
FROM employees;
```

first_name	last_name	salary	department_id	avg_department_salary
Jennifer	Whalen	4400.00	1	4400
Michael	Hartstein	13000.00	2	9500
Pat	Fay	6000.00	2	9500
Den	Raphaely	11000.00	3	4150
Alexander	Khoo	3100.00	3	4150
Shelli	Baida	2900.00	3	4150
Sigal	Tobias	2800.00	3	4150
Guy	Himuro	2600.00	3	4150
Karen	Colmenares	2500.00	3	4150
Susan	Mavris	6500.00	4	6500
Matthew	Weiss	8000.00	5	5886
Adam	Fripp	8200.00	5	5886
Payam	Kaufling	7900.00	5	5886
Shanta	Vollman	6500.00	5	5886
Irene	Mikkilineni	2700.00	5	5886
Sarah	Bell	4000.00	5	5886
Britney	Everett	3900.00	5	5886

Figure 2.2 Output of PARTITION BY clause.

UPDATE Command

UPDATE Query is used to modify the existing records in a table and use the WHERE clause with the UPDATE query to update the selected rows, otherwise all the rows would be affected.

Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2...., columnN = valueN
WHERE [condition];
```

Example

Update the ADDRESS for a customer whose ID number is 6 in the table.

```
UPDATE CUSTOMERS
SET ADDRESS = 'Pune'
WHERE ID = 6;
```

Modify all the ADDRESS and the SALARY column values in the CUSTOMERS table.

```
UPDATE CUSTOMERS
SET ADDRESS = 'Pune', SALARY = 1000.00;
```

DELETE Command

DELETE Query is used to delete the existing records from a table.

Syntax

```
DELETE FROM table_name
WHERE [condition];
```

Example

DELETE a customer, whose ID is 6.

```
DELETE FROM CUSTOMERS
WHERE ID = 6;
```

DELETE all the records from the CUSTOMERS table.

```
DELETE FROM CUSTOMERS;
```

2.1.3 Python

Python is an interpreted high-level general-purpose programming language. Its design philosophy emphasizes code readability with its use of significant indentation. Its language constructs as well as its object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects. The biggest strength of Python is huge collection of standard libraries which can be used for many works such as machine learning, GUI applications, web frameworks, image processing, web scraping, test frameworks, multimedia, scientific computing, text processing and etc.

The following are the libraries that are useful when going through this report:

Openpyxl

openpyxl is a Python library to read/write Excel 2010 xlsx/xlsm/xltx/xltm files. It allows the user to create, read and write spreadsheet. More importantly it can import data from spreadsheet and manipulate the data using Python data structures such as list, tuple, and dictionary.

Concurrent.Futures

concurrent.futures module is used to do multithreading and parallel programming in Python or to launch parallel tasks. The module provides different implementations using processes or threads.

Multiprocess

Tasks using the ProcessPoolExecutor spawn multiple processes (each process has its own Python interpreter), and by doing this, they bypass Python's global interpreter lock. Works best when there is a need to parallelize CPU-intensive tasks, as this takes advantage of multiple CPUs and cores.

Multithread

Tasks using the ThreadPoolExecutor use multiple threads in the same process. The threads share the same interpreter and memory space. Works best when there is a need to parallelize I/O-bound tasks or make responsive UIs because threads are not interruptible/killable like processes are.

Subprocess

A subprocess in Python is a task that a python script delegates to the Operative system (OS). The subprocess library allows us to execute and manage subprocesses directly from Python. That involves working with the standard input stdin, standard output stdout, and return codes.

2.1.4 Linux command

The Linux command is a utility of the Linux operating system. All basic and advanced tasks can be done by executing commands. The commands are executed on the Linux terminal. The terminal is a command-line interface to interact with the system, which is similar to the command prompt in the Windows OS.

Linux provides a powerful command-line interface compared to other operating systems such as Windows and MacOS. Both basic work and advanced work can be done through its terminal. Some basic tasks such as creating a file, deleting a file, moving a file, and more. In addition, advanced tasks such as administrative tasks (including package installation, user management), networking tasks (ssh connection), security tasks, and many more can also be performed.

Some of the basic Linux commands:

1. pwd command
2. mkdir command
3. rmdir command
4. ls command
5. cd command
6. cat command

pwd Command

The pwd command is used to display the location of the current working directory.

Syntax

```
pwd
```

Example

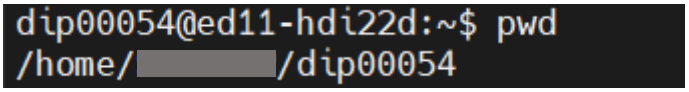


Figure 2.1 Example of pwd command

mkdir Command-

The mkdir command is used to create a new directory under any directory.

Syntax

```
mkdir <directory>
```

Example



Figure 2.2 Example of mkdir command

rmdir Command

The rmdir command is used to delete a directory.

Syntax

```
rmdir <directory>
```

Example

```
dip00054@ed11-hdi22d:~$ rmdir mew
dip00054@ed11-hdi22d:~$ ls mew
ls: cannot access 'mew': No such file or directory
```

Figure 2.3 Example of rmdir command

ls Command

The ls command is used to display a list of content of a directory.

Syntax

```
ls
```

Example

```
dip00054@ed11-hdi22d:~/mew$ ls
test.txt
```

Figure 2.4 Example of ls command

cd Command

The cd command is used to change the current directory.

Syntax

```
Cd <directory name>
```

Example

```
dip00054@ed11-hdi22d:~$ cd mew
dip00054@ed11-hdi22d:~/mew$ pwd
/home/[redacted]/dip00054/mew
```

Figure 2.5 Example of cd command

cat Command

The cat command is used to create a file, display content of the file, copy the content of one file to another file, and more.

Syntax

Create and write on a file

```
cat> <File_Name>
//Enter file content
```

Display content of the file

```
Cat <File_name>
```

Example

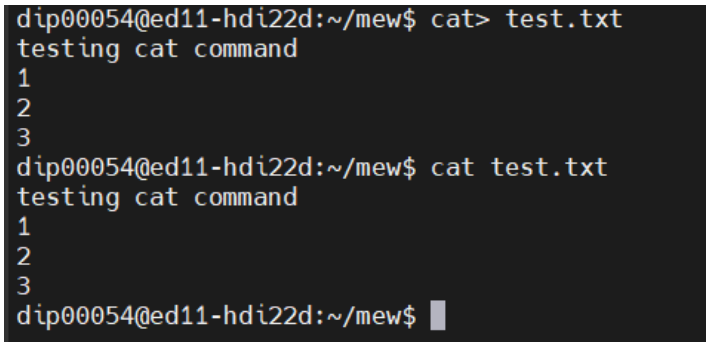


Figure 2.6 Example of cat command

2.2 Technologies used in practice

2.2.1 SourceTree

SourceTree provides a graphical interface for repositories, between users and Git. It allows the user to manage all the GIT repositories both hosted or local through a single client, and use git commands such as commit, push, pull and merge without coding. It can be connected with the repositories in Bitbucket, Stash, Microsoft TFS or GitHub.

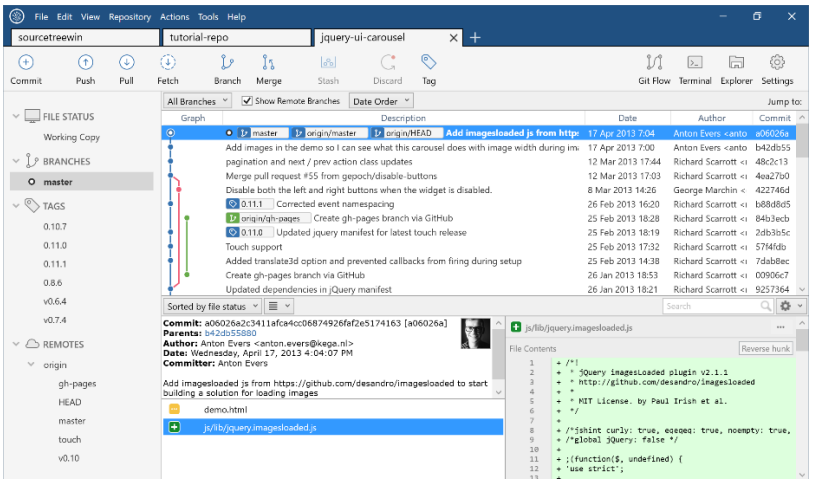


Figure 2.7 Screenshot of SSMS user interface.

2.2.2 MobaXterm

MobaXterm is an enhanced terminal for Windows and a toolbox for remote computing. It provides important remote network tools such as SSH, X11, RDP, VNC, FTP, MOSH, and etc. and Unix commands such as bash, ls, cat, sed, grep, awk, rsync, and etc. to Windows desktop, in a single portable exe file.

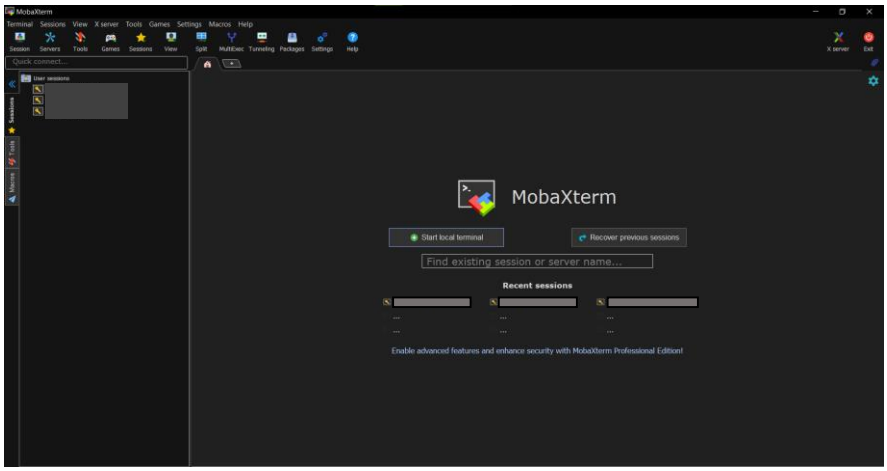


Figure 2.8 Screenshot of MobaXterm window screen.

2.2.3 Jenkins

Jenkins is an open-source automation tool written in Java with plugins built for Continuous Integration purposes. Jenkins is used to build and test software projects continuously making it easier for developers to integrate changes to the project and making it easier for users to obtain a fresh build. It also allows you to continuously deliver your software by integrating with a large number of testing and deployment technologies.

With Jenkins, organizations can accelerate the software development process through automation. Jenkins integrates development life-cycle processes of all kinds, including build, document, test, package, stage, deploy, static analysis, and much more.

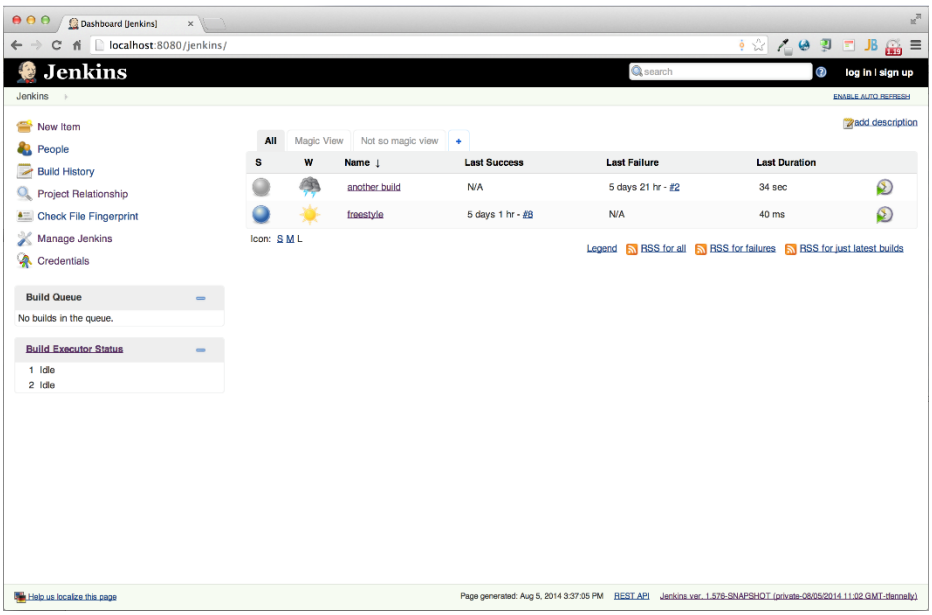


Figure 2.9 Screenshot of Jenkins user interface

2.2.4 Visual Studio Code

Visual Studio Code is an Integrated Development Environment (IDE) made by Microsoft for Windows, Linux and macOS. It is a source-code editor that can be used with a variety of programming languages, including Java, JavaScript, Go, Node.js, Python and C++. Features include support for

debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded Git. Users can change the theme, keyboard shortcuts, preferences, and install extensions that add additional functionality.

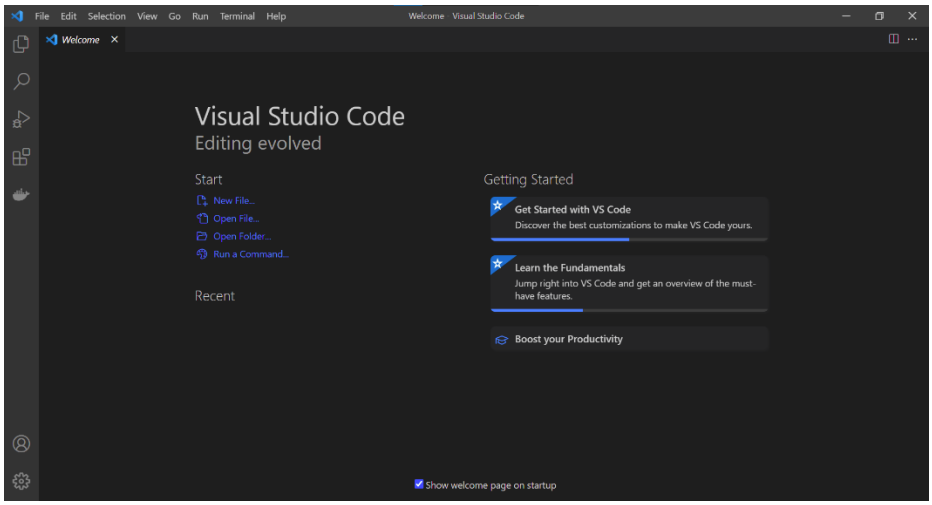


Figure 2.10 Screenshot of Visual Studio Code interface.

2.2.5 MS Excel

Excel is a spreadsheet application developed and published by Microsoft. Unlike a word processor, such as Microsoft Word, Excel organizes data in columns and rows. Rows and columns intersect at a space called a cell. Each cell can contain a single of data, such as text, a numerical value, or a formula.

It is a useful tool for organizing and performing calculations on data. It can analyze data, calculate statistics, generate pivot tables, and represent data as a chart or graph.

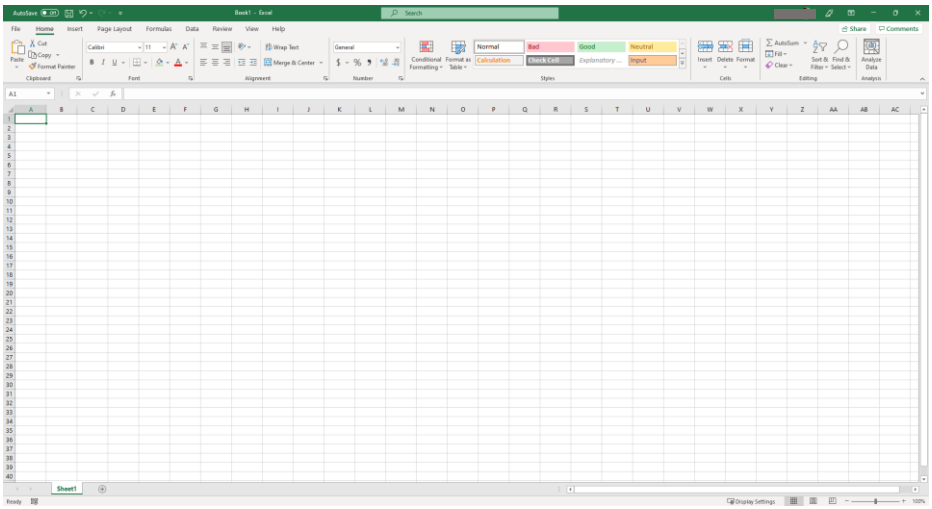


Figure 2.11 Screenshot of Excel interface.

Chapter 3

Work Plan and Procedures

For this co-operative program, the position of the work is data engineering analyst(?) assigned to the framework team of a Data Lake Upgrade project. This project is an on-going project, wherein the responsibility varies according to the project timeline. The detail of the work that are done will be discussed in this chapter.

3.1 Plans of the Project

Table 3.1 Plans of the project

Topics	2021																			
	Jun				Jul				Aug				Sept				Oct			
Jenkins Deployment Pipeline																				
Automate Deployment Pipeline (Jenkins) test																				
Jenkins Deployment Pipeline transition																				
Production Preparation																				
Validate file with inventory and compare last modified date of file																				
Validate table & view inventory																				
Replace new edge node IP of impacted ADF Pipeline																				
Validate date function																				
Validate HDI4.0 insert overwrite with empty result																				
Validate HDI4.0 drop partition with varchar datatype																				
Work Transition																				

The table 3.1 shows the work plan in total of 22 weeks. Wherein, the first two months was mainly assigned to test Jenkins Development Pipeline and the last three months was assigned with tasks for the project's deployment phase preparation.

3.2 Detailed Procedures of the Project

This part will explain the background of the project and detail of each work assigned which are Jenkins Deployment Pipeline and Production Preparation.

3.2.1 Project Background

Data Lake Upgrade (DL Upgrade) project, the objective of the project is quite straightforwardly explained by its name, which is to upgrade the data lake. The current data lake platform used is Microsoft Azure HDInsight version 3.6 (HDI3.6). The motivation of this project is that as-is version is soon to be outdated or will no longer be supported by the provider in the near future. Other from that, the users can utilize better technology and optimized features as an advantage of this upgrading, which is currently to be version 4.0 (HDI4.0).

However, moving from HDI3.6 to 4.0 is a major change. The current data lake framework does not meet the current industry standards (HDI4.0 and Python3) leading to the malfunctioning of the process. Syntax and commands that works on HDI 3.6 does not work the same way as 4.0 does. Wherefore, the DL Upgrade project will work on modernizing the framework along with enhancing the current deployment process (SDLC) which is now using the Jenkins Deployment Pipeline, develops script to automate convert vast amount of jobs (HDI3.6 syntax/commands to align with 4.0's), migrates framework's data to new database, and besides, takes care of the activities management.

3.2.2 Jenkins Deployment Pipeline

The Jenkins Deployment Pipeline is used for deploying codes, scripts or configurations from development environment to pre-production or production environment. In the as-is deployment process, the developer prepares deployment scripts and documents, send them to deployment team, in case the

developer failed to pass the quality control, the developer will be required to send them again which might take up to weeks until they can send them again depending to the opening and closing of deploying service. This takes so much effort for the developer and a great deal of turnaround time. The enhancing of Jenkins Deployment Pipeline helps ease the pain-point in quality validation process.

I was assigned to help with unit testing and the testing of the Jenkins Deployment Pipeline on the development environment and some on pre-production environment. Testing each functions according to the test scenarios, record and capture the results of each test scenarios on excel and preparing documents and slides for the transition sessions.

3.2.3 Production Preparation

Production preparation is the stage where we make sure that the new environment is ready for the production phase. There is a checklist of things we have to do or check, such as to check the readiness of the tables that are created, the libraries installed, configurations and back up works. The tasks in the checklist were distributed to the members of the team. We prepare automation scripts for each task to avoid manual work. The scripts will be combined after all are done, which will allow us to perform a production preparation tasks in the checklist at one go.

I was assigned to implement Python code to perform the task and made a transition of my work to the team for further maintenance when combining all the scripts together.

3.3 Practical Procedures Used for the Project

In this part will be explaining the practical step and work that has been performed in the project.

3.3.1 Practical Procedures of Jenkins Deployment Pipeline

3.3.1.1 Automate Deployment Pipeline (Jenkins) Test

Understanding the how each function and process of the pipeline works, testing the Jenkins Deployment Pipeline according to the test scenarios given, identifying any bugs or errors and report to the developers,

collecting test result by documenting, and capturing the output screenshots. The test result will be sending to the client to check and request for confirmation.

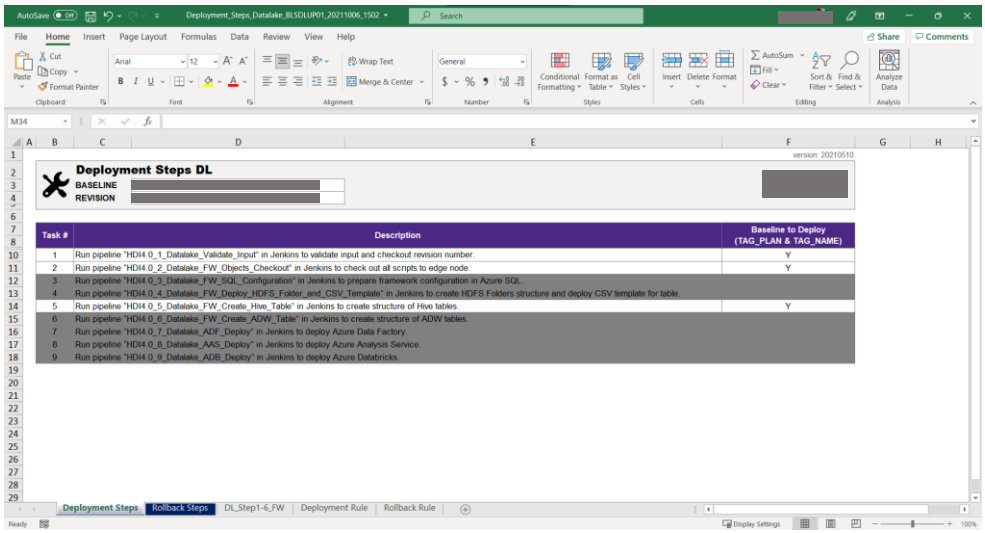


Figure 3.1 Preparing Input Excel Files.

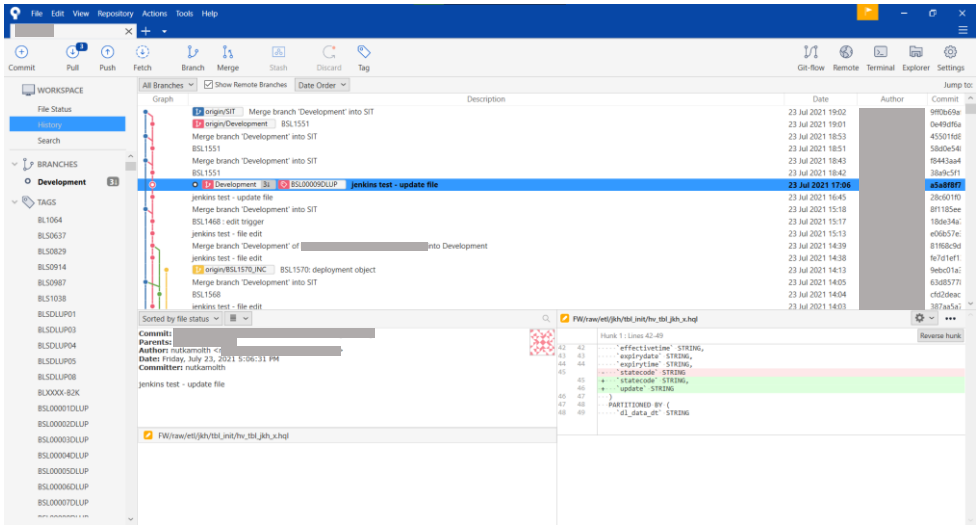


Figure 3.2 Commit files used in testing to GIT.

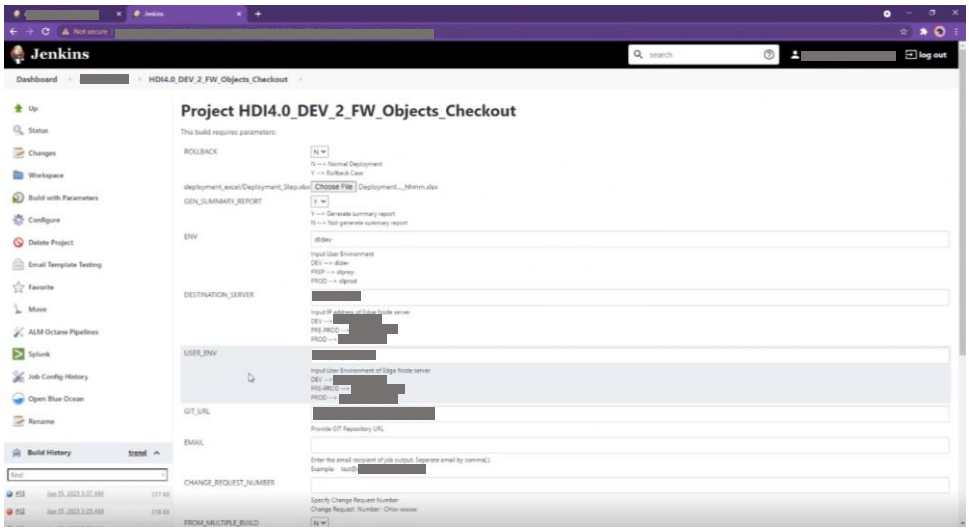


Figure 3.3 Run pipeline on Jenkins.

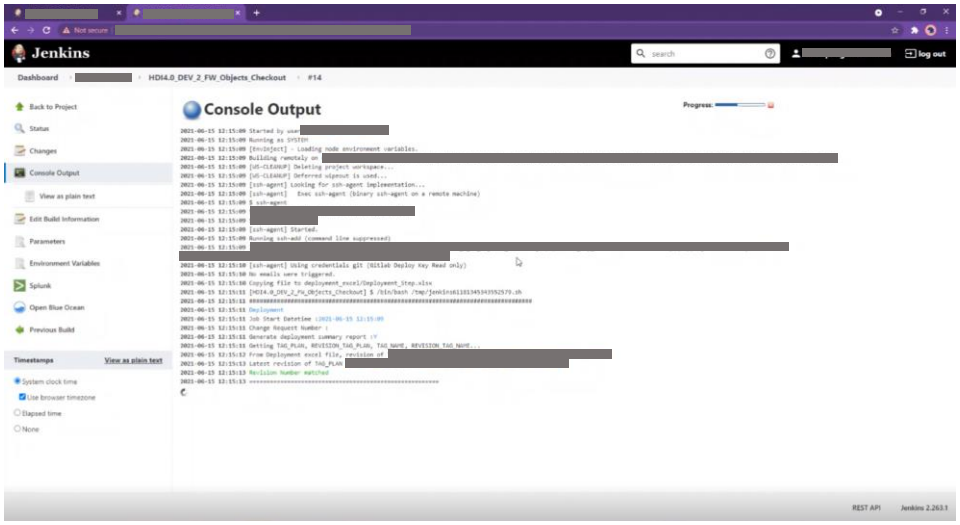


Figure 3.4 Jenkins generate log on console output.

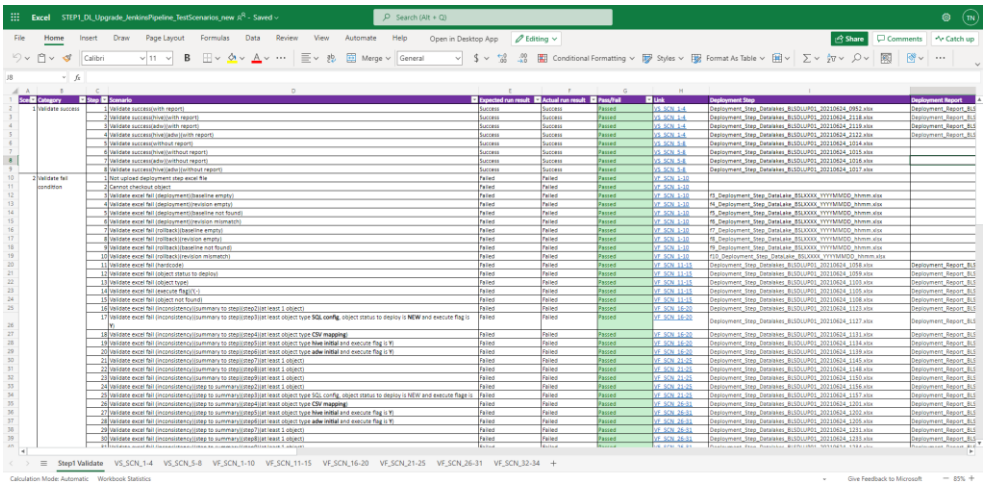


Figure 3.5 Jenkins test excel, the worksheet shows test scenarios, test result status, link to result screen capturing and input and output file name.

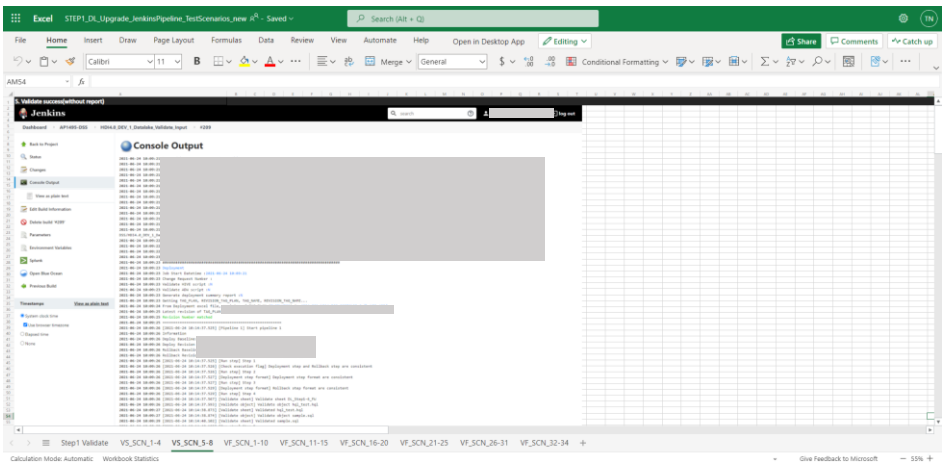


Figure 3.6 Jenkins test excel, the worksheet shows the screen capture of Jenkins console output on each scenario.

3.3.1.2 Jenkins Deployment Pipeline Transition

After testing the Jenkins Deployment Pipeline and received the client's acceptance, transitions session is scheduled to deliver the work to the client. The work assigned in this task are to help preparing documents, training material, presentation slides of Jenkins Deployment Pipeline transition. The session was conducted

to explain the workings, usage, demonstrate on how to deploy the Jenkins Deployment Pipeline and clarifies queries from the clients.

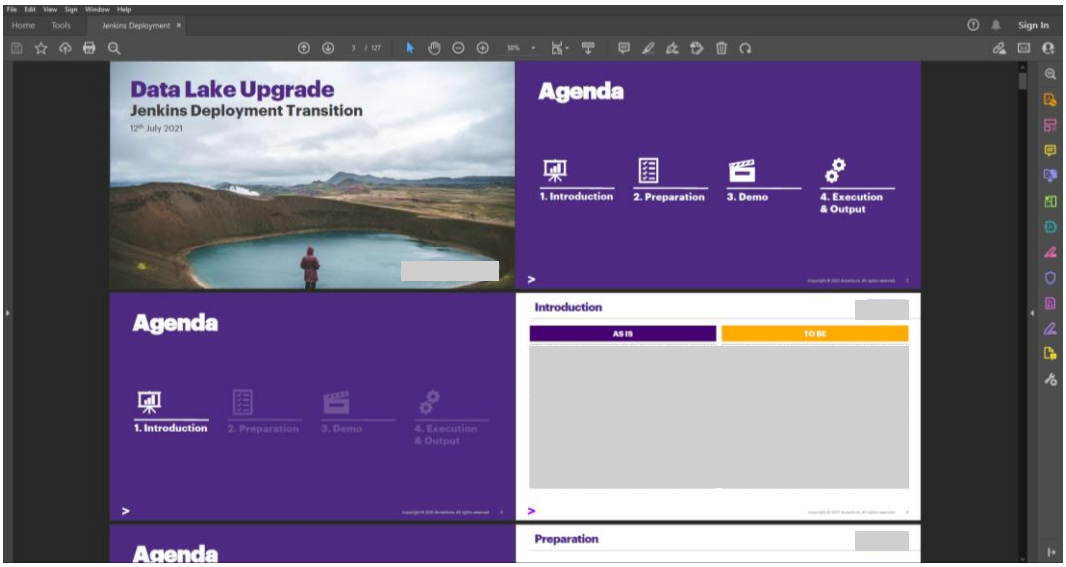


Figure 3.7 Jenkins Deployment Transition Presentation slides

3.3.2 Practical Procedures of Production Preparation

3.3.2.1 Validate file with inventory and compare last modified date of file

The is a process of file validation to ensure that all the files are present on the new environment before go-live, by checking with the inventory which is the list of all the files to be deploy. In addition of file availability, it compares the last modified date of the file to identify the version of the file.

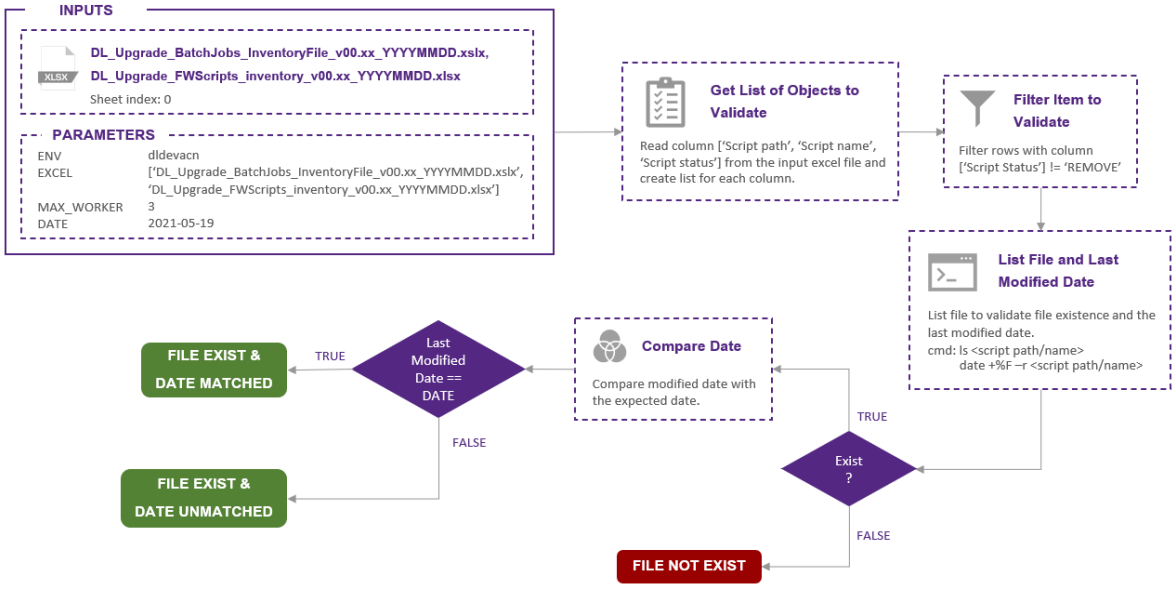
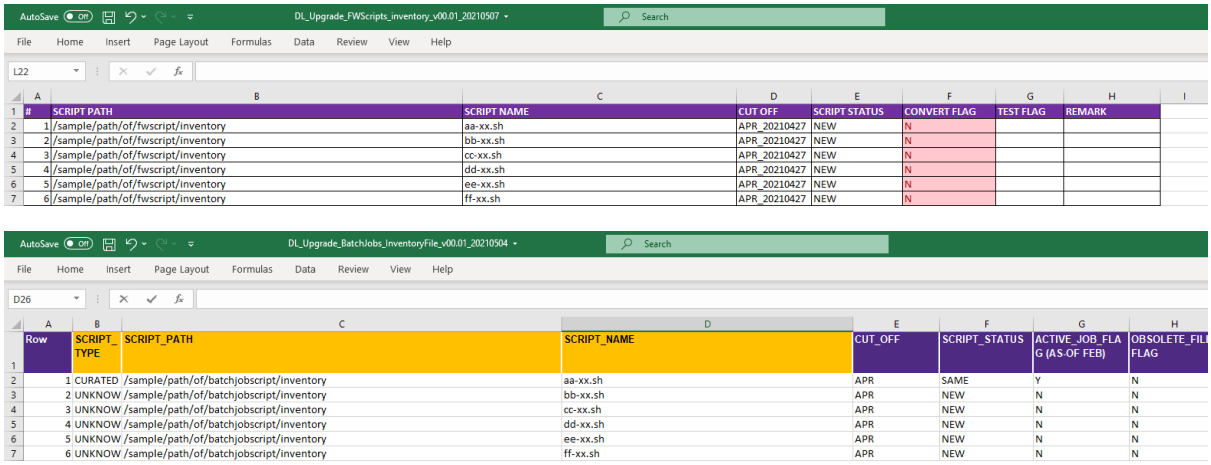


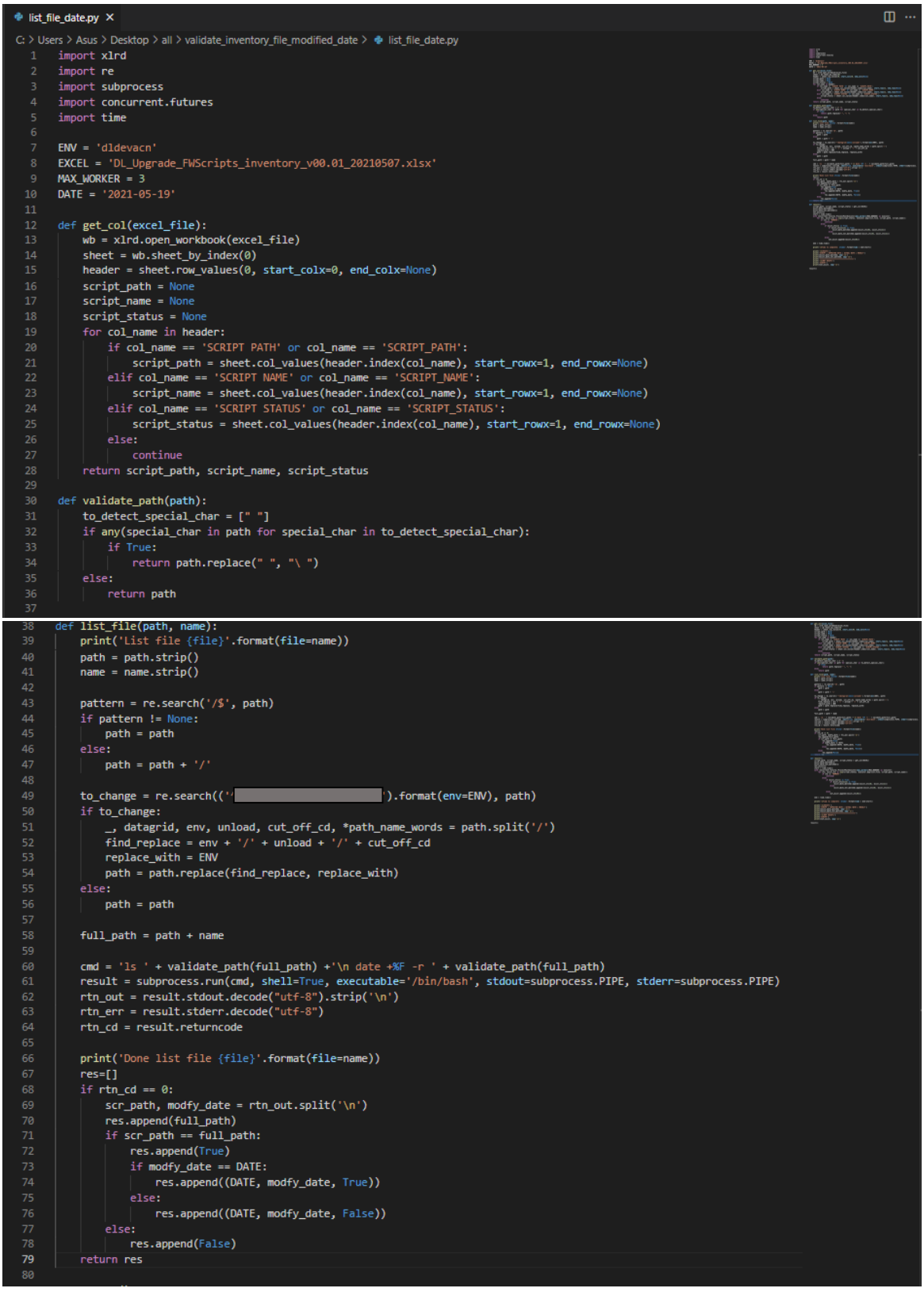
Figure 3.8 Workflow chart of validating file with inventory and compare last modified date.



#	SCRIPT PATH	SCRIPT NAME	CUT OFF	SCRIPT STATUS	CONVERT FLAG	TEST FLAG	REMARK
1	/sample/path/of/fwscript/inventory	aa-xx.sh	APR_20210427	NEW	N		
2	/sample/path/of/fwscript/inventory	bb-xx.sh	APR_20210427	NEW	N		
3	/sample/path/of/fwscript/inventory	cc-xx.sh	APR_20210427	NEW	N		
4	/sample/path/of/fwscript/inventory	dd-xx.sh	APR_20210427	NEW	N		
5	/sample/path/of/fwscript/inventory	ee-xx.sh	APR_20210427	NEW	N		
6	/sample/path/of/fwscript/inventory	ff-xx.sh	APR_20210427	NEW	N		

Row	SCRIPT_TYPE	SCRIPT_PATH	SCRIPT_NAME	CUT_OFF	SCRIPT_STATUS	ACTIVE_JOB_FLAG (AS-OF FEB)	OBSOLETE_FILE FLAG
1	CURATED	/sample/path/of/batchjobscript/inventory	aa-xx.sh	APR	SAME	Y	N
2	UNKNOW	/sample/path/of/batchjobscript/inventory	bb-xx.sh	APR	NEW	N	N
3	UNKNOW	/sample/path/of/batchjobscript/inventory	cc-xx.sh	APR	NEW	N	N
4	UNKNOW	/sample/path/of/batchjobscript/inventory	dd-xx.sh	APR	NEW	N	N
5	UNKNOW	/sample/path/of/batchjobscript/inventory	ee-xx.sh	APR	NEW	N	N
6	UNKNOW	/sample/path/of/batchjobscript/inventory	ff-xx.sh	APR	NEW	N	N

Figure 3.9 Screenshot of sample script inventory excel input file.



```
list_file_date.py
C: > Users > Asus > Desktop > all > validate_inventory_file_modified_date > list_file_date.py
import xlrd
import re
import subprocess
import concurrent.futures
import time

ENV = 'dldevacn'
EXCEL = 'DL_Upgrade_FWScripts_inventory_v00.01_20210507.xlsx'
MAX_WORKER = 3
DATE = '2021-05-19'

def get_col(excel_file):
    wb = xlrd.open_workbook(excel_file)
    sheet = wb.sheet_by_index(0)
    header = sheet.row_values(0, start_colx=0, end_colx=None)
    script_path = None
    script_name = None
    script_status = None
    for col_name in header:
        if col_name == 'SCRIPT PATH' or col_name == 'SCRIPT_PATH':
            script_path = sheet.col_values(header.index(col_name), start_rowx=1, end_rowx=None)
        elif col_name == 'SCRIPT NAME' or col_name == 'SCRIPT_NAME':
            script_name = sheet.col_values(header.index(col_name), start_rowx=1, end_rowx=None)
        elif col_name == 'SCRIPT STATUS' or col_name == 'SCRIPT_STATUS':
            script_status = sheet.col_values(header.index(col_name), start_rowx=1, end_rowx=None)
        else:
            continue
    return script_path, script_name, script_status

def validate_path(path):
    to_detect_special_char = [" "]
    if any(special_char in path for special_char in to_detect_special_char):
        if True:
            return path.replace(" ", "\ ")
    else:
        return path

def list_file(path, name):
    print('List file {file}'.format(file=name))
    path = path.strip()
    name = name.strip()

    pattern = re.search('/$', path)
    if pattern != None:
        path = path
    else:
        path = path + '/'

    to_change = re.search('[redacted]').format(env=ENV), path)
    if to_change:
        _, datagrid, env, unload, cut_off_cd, *path_name_words = path.split('/')
        find_replace = env + '/' + unload + '/' + cut_off_cd
        replace_with = ENV
        path = path.replace(find_replace, replace_with)
    else:
        path = path

    full_path = path + name

    cmd = 'ls ' + validate_path(full_path) +'\n date +%F -r ' + validate_path(full_path)
    result = subprocess.run(cmd, shell=True, executable='/bin/bash', stdout=subprocess.PIPE, stderr=subprocess.PIPE)
    rtn_out = result.stdout.decode("utf-8").strip('\n')
    rtn_err = result.stderr.decode("utf-8")
    rtn_cd = result.returncode

    print('Done list file {file}'.format(file=name))
    res=[]
    if rtn_cd == 0:
        scr_path, modfy_date = rtn_out.split('\n')
        res.append(full_path)
        if scr_path == full_path:
            res.append(True)
            if modfy_date == DATE:
                res.append((DATE, modfy_date, True))
            else:
                res.append((DATE, modfy_date, False))
        else:
            res.append(False)
    return res
```

Figure 3.10 Screenshot of validate file with inventory and compare last modified date of file code

```python
def result():
    script_path, script_name, script_status = get_col(EXCEL)
    exist_date_matched=[]
    exist_date_not_matched=[]
    not_exist=[]
    start = time.time()
    with concurrent.futures.ProcessPoolExecutor(max_workers=MAX_WORKER) as executor:
        for sts, exist_sts in zip(script_status, executor.map(list_file, script_path, script_name)):
            if sts == 'REMOVE':
                continue
            else:
                if exist_sts[1] is True:
                    if exist_sts[2][2] is True:
                        exist_date_matched.append((exist_sts[0], exist_sts[2]))
                    else:
                        exist_date_not_matched.append((exist_sts[0], exist_sts[2]))

                else:
                    not_exist.append((exist_sts[0]))

    end = time.time()

    print('\nTime to complete: {time}'.format(time = end-start))

    print('\n[EXIST]')
    print('SCRIPT | EXPECTED DATE | ACTUAL DATE | RESULT')
    print(*exist_date_matched, sep='\n')
    print(*exist_date_not_matched, sep='\n')
    print('==================================')
    print('\n[NOT EXIST]')
    print('SCRIPT')
    print(*not_exist, sep='\n')

result()
```

Figure 3.11 Screenshot of validate file with inventory and compare last modified date of file code (cont.)

```
dip00054@ed11-hdi22d:/[redacted]validate_inventory_file_modified_date$ python3 list_file_date.py
[INVENTORY] DL_Upgrade_FWScripts_inventory_v00.01_20210507.xlsx
ls [redacted]
ls [redacted]
ls [redacted]
Done ls [redacted]
ls [redacted]
Done ls [redacted]
ls [redacted]
Done ls [redacted]
ls [redacted]
Done ls [redacted]
ls [redacted]
Done ls [redacted]
ls [redacted]
Done ls [redacted]
ls [redacted]
Done ls [redacted]
ls [redacted]
Done ls [redacted]
ls [redacted]
Done ls [redacted]
Done ls [redacted]

Time to complete: 5.9692981243133545
[EXIST]
SCRIPT | EXPECTED DATE | ACTUAL DATE | RESULT
([redacted].sh', ('2021-05-19', '2021-05-19', True))
([redacted].sh', ('2021-05-19', '2021-05-19', True))
([redacted].sh', ('2021-05-19', '2021-05-19', True))
([redacted].sh', ('2021-05-19', '2021-05-19', True))
([redacted].sh', ('2021-05-19', '2021-05-19', True))
([redacted].sh', ('2021-05-19', '2021-05-19', True))
([redacted]', '2021-05-19', True))
```

Figure 3.12 Screenshot of validate file with inventory and compare last modified date of file output.

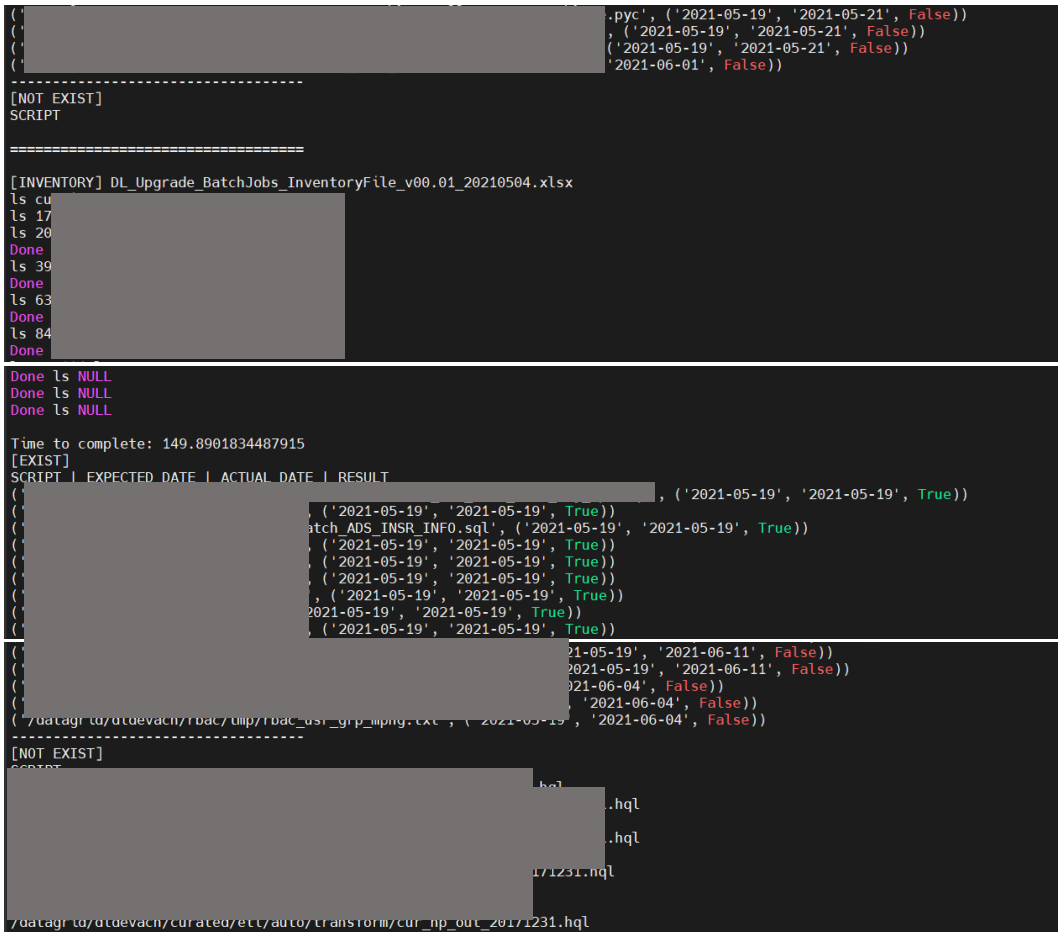


Figure 3.13 Screenshot of validate file with inventory and compare last modified date of file output (cont.)

3.3.2.2 Validate table & view inventory

Process of validating table and view to ensure that all the tables and views are present in the new environment before production phase by checking with the inventory which is the list of all the tables and views that are expected to be present.

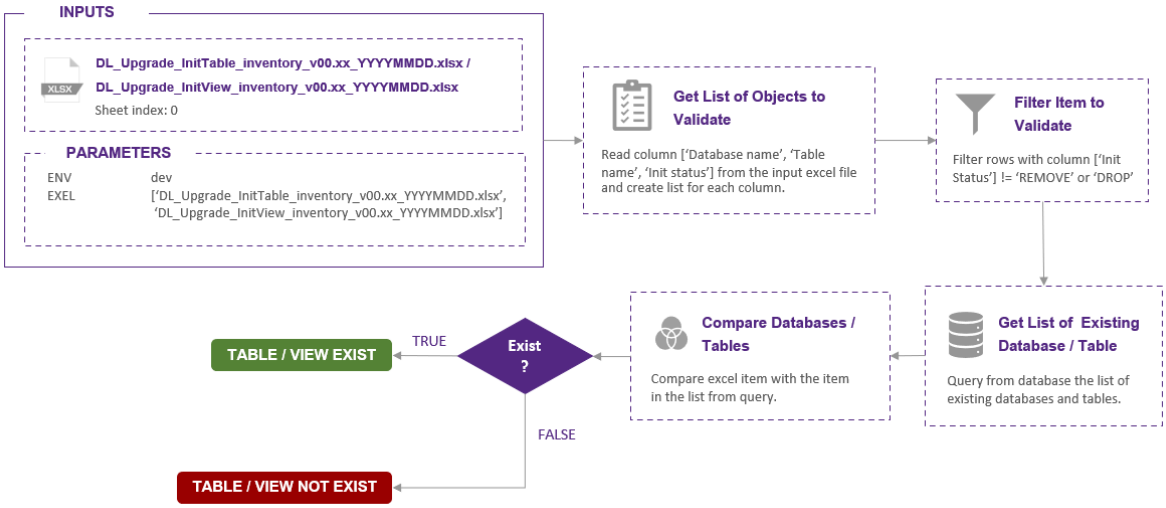
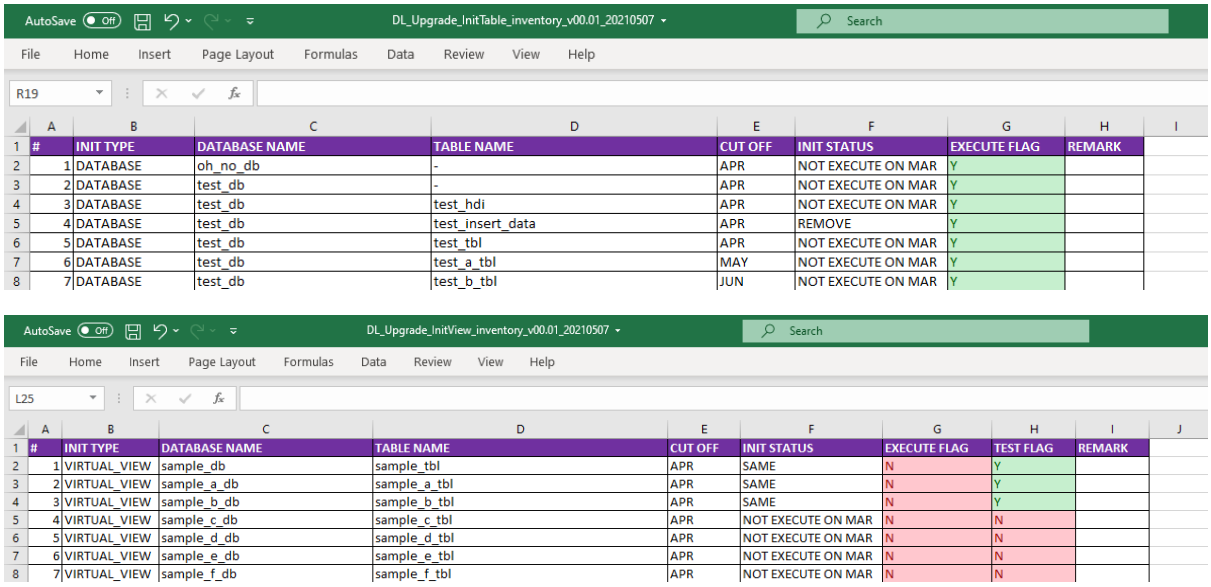


Figure 3.14 Workflow chart of validating table and view inventory.



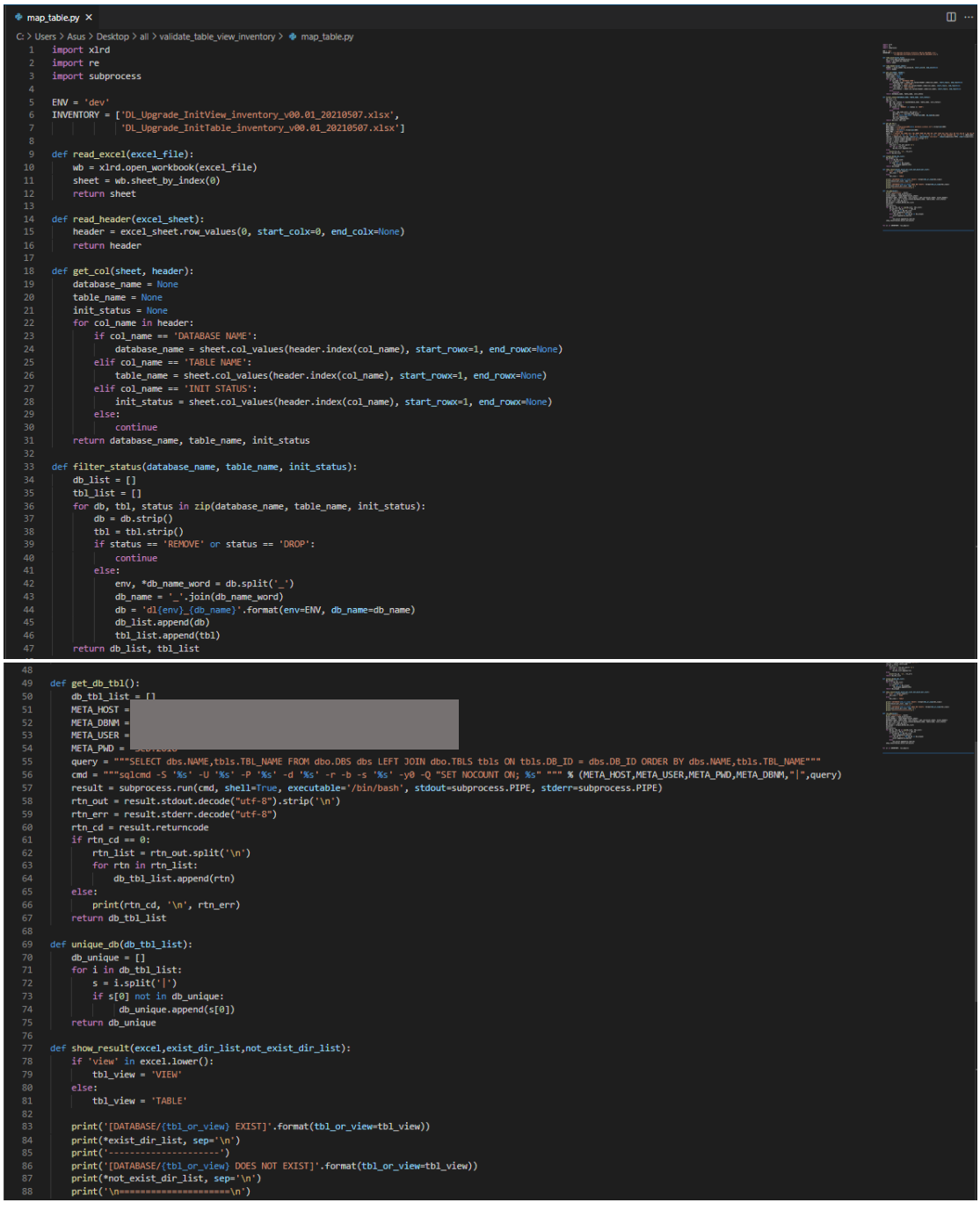
DL_Upgrade_InitTable_inventory_v00.01_20210507

#	INIT TYPE	DATABASE NAME	TABLE NAME	CUT OFF	INIT STATUS	EXECUTE FLAG	REMARK
1	DATABASE	oh_no_db	-	APR	NOT EXECUTE ON MAR	Y	
2	DATABASE	test_db	-	APR	NOT EXECUTE ON MAR	Y	
3	DATABASE	test_db	test_hdi	APR	NOT EXECUTE ON MAR	Y	
4	DATABASE	test_db	test_insert_data	APR	REMOVE	Y	
5	DATABASE	test_db	test_tbl	APR	NOT EXECUTE ON MAR	Y	
6	DATABASE	test_db	test_a_tbl	MAY	NOT EXECUTE ON MAR	Y	
7	DATABASE	test_db	test_b_tbl	JUN	NOT EXECUTE ON MAR	Y	

DL_Upgrade_InitView_inventory_v00.01_20210507

#	INIT TYPE	DATABASE NAME	TABLE NAME	CUT OFF	INIT STATUS	EXECUTE FLAG	TEST FLAG	REMARK
1	VIRTUAL_VIEW	sample_db	sample_tbl	APR	SAME	N	Y	
2	VIRTUAL_VIEW	sample_a_db	sample_a_tbl	APR	SAME	N	Y	
3	VIRTUAL_VIEW	sample_b_db	sample_b_tbl	APR	SAME	N	Y	
4	VIRTUAL_VIEW	sample_c_db	sample_c_tbl	APR	NOT EXECUTE ON MAR	N	N	
5	VIRTUAL_VIEW	sample_d_db	sample_d_tbl	APR	NOT EXECUTE ON MAR	N	N	
6	VIRTUAL_VIEW	sample_e_db	sample_e_tbl	APR	NOT EXECUTE ON MAR	N	N	
7	VIRTUAL_VIEW	sample_f_db	sample_f_tbl	APR	NOT EXECUTE ON MAR	N	N	

Figure 3.15 Screenshot of table and view inventory input excel file.



```python
import xlrd
import re
import subprocess

ENV = 'dev'
INVENTORY = ['DL_Upgrade_InitView_inventory_v00.01_20210507.xlsx',
             'DL_Upgrade_InitTable_inventory_v00.01_20210507.xlsx']

def read_excel(excel_file):
    wb = xlrd.open_workbook(excel_file)
    sheet = wb.sheet_by_index(0)
    return sheet

def read_header(excel_sheet):
    header = excel_sheet.row_values(0, start_colx=0, end_colx=None)
    return header

def get_col(sheet, header):
    database_name = None
    table_name = None
    init_status = None
    for col_name in header:
        if col_name == 'DATABASE NAME':
            database_name = sheet.col_values(header.index(col_name), start_rowx=1, end_rowx=None)
        elif col_name == 'TABLE NAME':
            table_name = sheet.col_values(header.index(col_name), start_rowx=1, end_rowx=None)
        elif col_name == 'INIT STATUS':
            init_status = sheet.col_values(header.index(col_name), start_rowx=1, end_rowx=None)
        else:
            continue
    return database_name, table_name, init_status

def filter_status(database_name, table_name, init_status):
    db_list = []
    tbl_list = []
    for db, tbl, status in zip(database_name, table_name, init_status):
        db = db.strip()
        tbl = tbl.strip()
        if status == 'REMOVE' or status == 'DROP':
            continue
        else:
            env, *db_name_word = db.split('_')
            db_name = '_'.join(db_name_word)
            db = 'dl{env}_{db_name}'.format(env=ENV, db_name=db_name)
            db_list.append(db)
            tbl_list.append(tbl)
    return db_list, tbl_list


def get_db_tbl():
    db_tbl_list = []
    META_HOST = [redacted]
    META_DBNM =
    META_USER =
    META_PWD =
    query = """SELECT dbs.NAME,tbls.TBL_NAME FROM dbo.DBS dbs LEFT JOIN dbo.TBLS tbls ON tbls.DB_ID = dbs.DB_ID ORDER BY dbs.NAME,tbls.TBL_NAME"""
    cmd = """sqlcmd -S '%s' -U '%s' -P '%s' -d '%s' -r -b -s '%s' -y0 -Q "SET NOCOUNT ON; %s" """ % (META_HOST,META_USER,META_PWD,META_DBNM,"|",query)
    result = subprocess.run(cmd, shell=True, executable='/bin/bash', stdout=subprocess.PIPE, stderr=subprocess.PIPE)
    rtn_out = result.stdout.decode("utf-8").strip('\n')
    rtn_err = result.stderr.decode("utf-8")
    rtn_cd = result.returncode
    if rtn_cd == 0:
        rtn_list = rtn_out.split('\n')
        for rtn in rtn_list:
            db_tbl_list.append(rtn)
    else:
        print(rtn_cd, '\n', rtn_err)
    return db_tbl_list

def unique_db(db_tbl_list):
    db_unique = []
    for i in db_tbl_list:
        s = i.split('|')
        if s[0] not in db_unique:
            db_unique.append(s[0])
    return db_unique

def show_result(excel,exist_dir_list,not_exist_dir_list):
    if 'view' in excel.lower():
        tbl_view = 'VIEW'
    else:
        tbl_view = 'TABLE'

    print('[DATABASE/{tbl_or_view} EXIST]'.format(tbl_or_view=tbl_view))
    print(*exist_dir_list, sep='\n')
    print('--------------------')
    print('[DATABASE/{tbl_or_view} DOES NOT EXIST]'.format(tbl_or_view=tbl_view))
    print(*not_exist_dir_list, sep='\n')
    print('\n====================\n')
```

Figure 3.16 Screenshot of validating table and view inventory code.

```
89
90  def run_map(excel):
91      print('[Inventory]', excel)
92      excel_sheet = read_excel(excel)
93      excel_header = read_header(excel_sheet)
94      database_name, table_name, init_status = get_col(excel_sheet, excel_header)
95      db_list, tbl_list = filter_status(database_name, table_name, init_status)
96      db_tbl_list = get_db_tbl()
97      db_unique = unique_db(db_tbl_list)
98      exist=[]
99      not_exist=[]
100     for db_nm, tbl_nm in zip(db_list, tbl_list):
101         to_search = db_nm + '|' + tbl_nm
102         if to_search in db_tbl_list:
103             exist.append(to_search)
104         elif tbl_nm == '-' and db_nm in db_unique:
105             exist.append(to_search)
106         else:
107             not_exist.append(to_search)
108     show_result(excel,exist,not_exist)
109
110
111 for xl in INVENTORY: run_map(xl)
112
```

Figure 3.17 Screenshot of validating table and view inventory code (cont.)

```
dip00054@ed11-hdi22d:/[redacted]/validate_table_view_inventory$ python3 map_table.py
[Inventory] DL_Upgrade_InitView_inventory_v00.01_20210507.xlsx
[DATABASE/VIEW EXIST]
[redacted]
----------
[DATABASE/VIEW DOES NOT EXIST]
[redacted]
==========

[Inventory] DL_Upgrade_InitTable_inventory_v00.01_20210507.xlsx
[DATABASE/TABLE EXIST]
[redacted]
```

Figure 3.18 Screenshot of validating table and view inventory output



Figure 3.19 Screenshot of validating table and view inventory output (cont.)

3.3.2.3 Replace new edge node IP (as is -> to be) of impacted ADF Pipeline

The objective of this task is to change the IP address in the files to support the changes of IP address in the future. The process of this task is divided into two subtasks which are 1. Identifying all the current IP addresses, 2. Replacing current IP addresses in each file with the new IP addresses, in which the file of the new version will be generated separately.



Figure 3.20 Workflow of finding as is distinct IP Address.

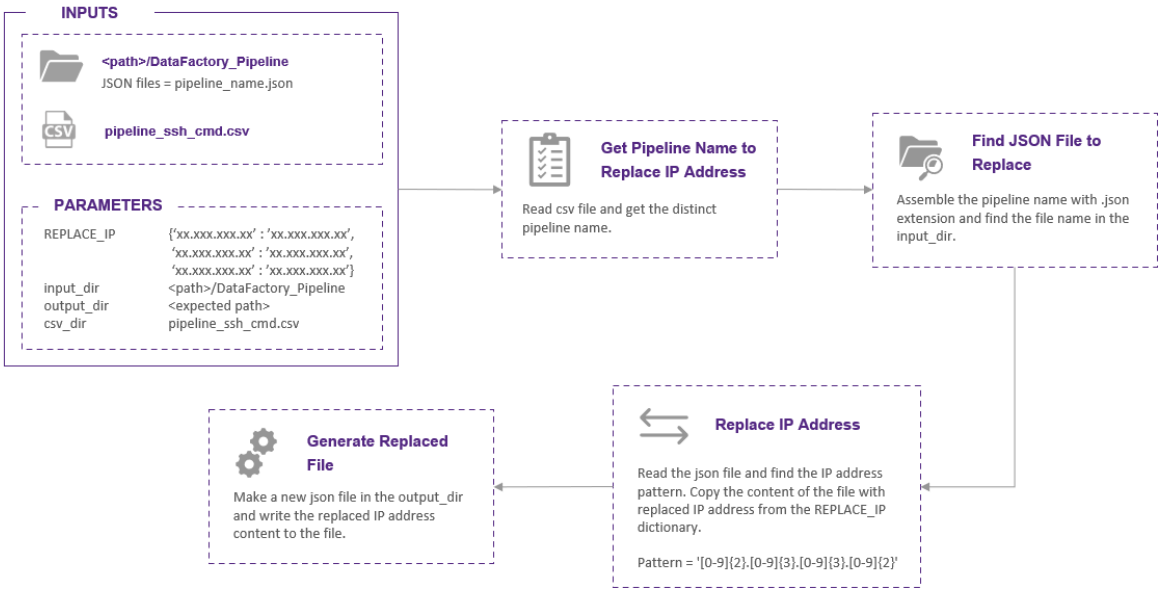


Figure 3.21 Workflow of replacing IP address.

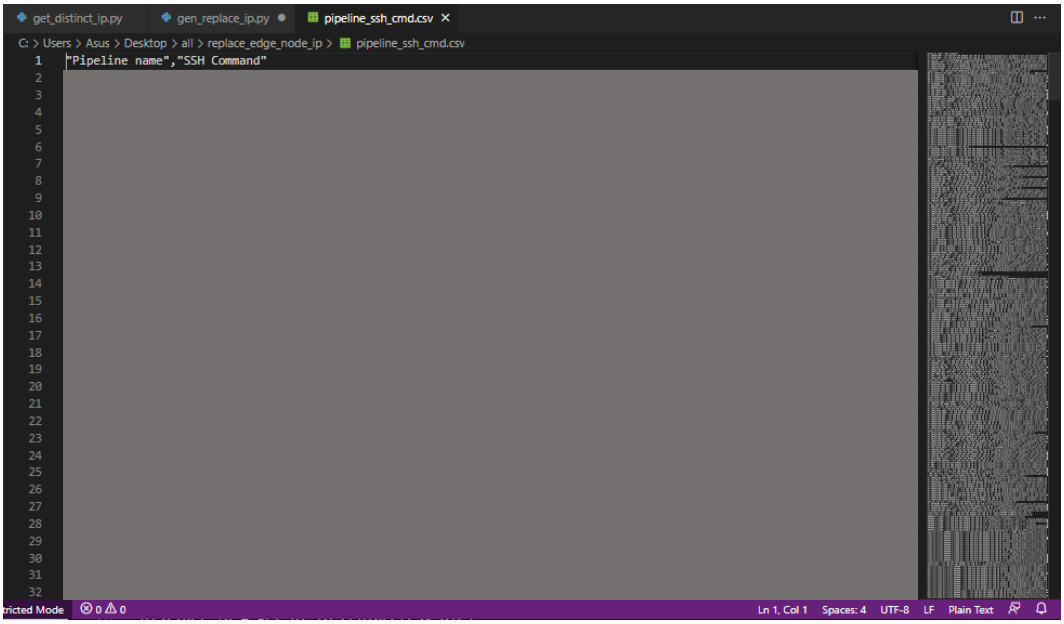


Figure 3.22 Screenshot of inut csv file.

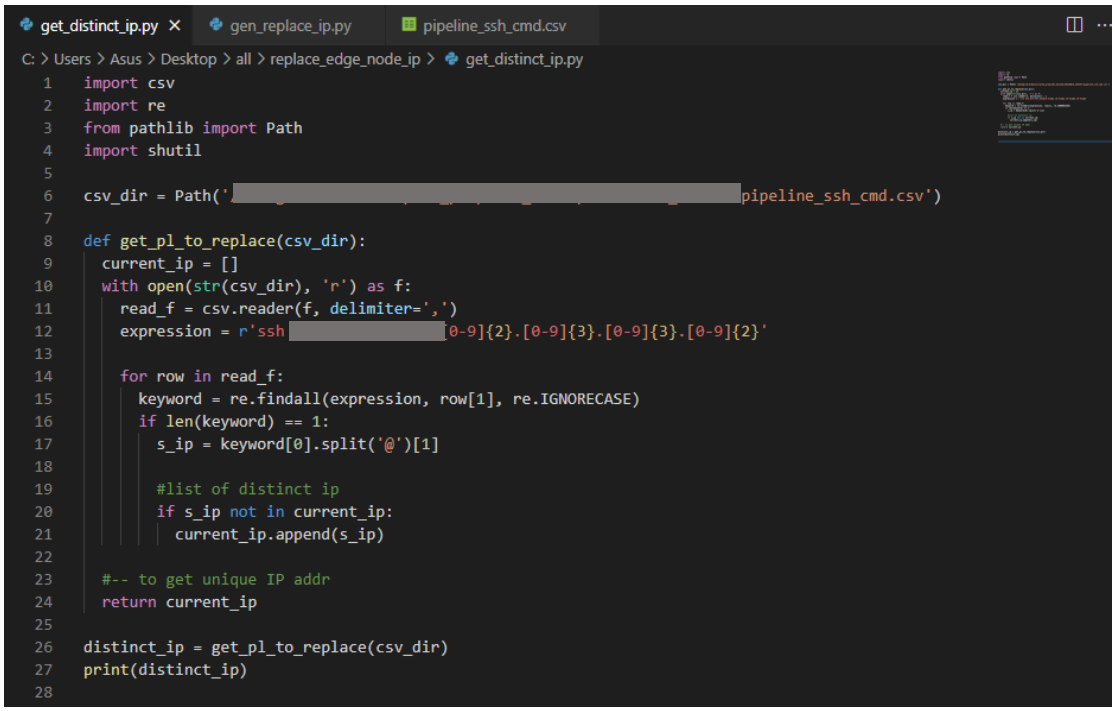


Figure 3.23 Screenshot of getting a distinct IP address code.

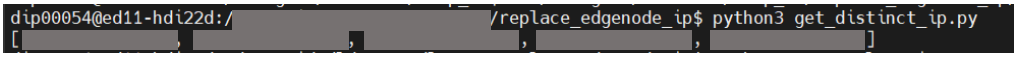


Figure 3.24 Screenshot of getting a distinct IP address output

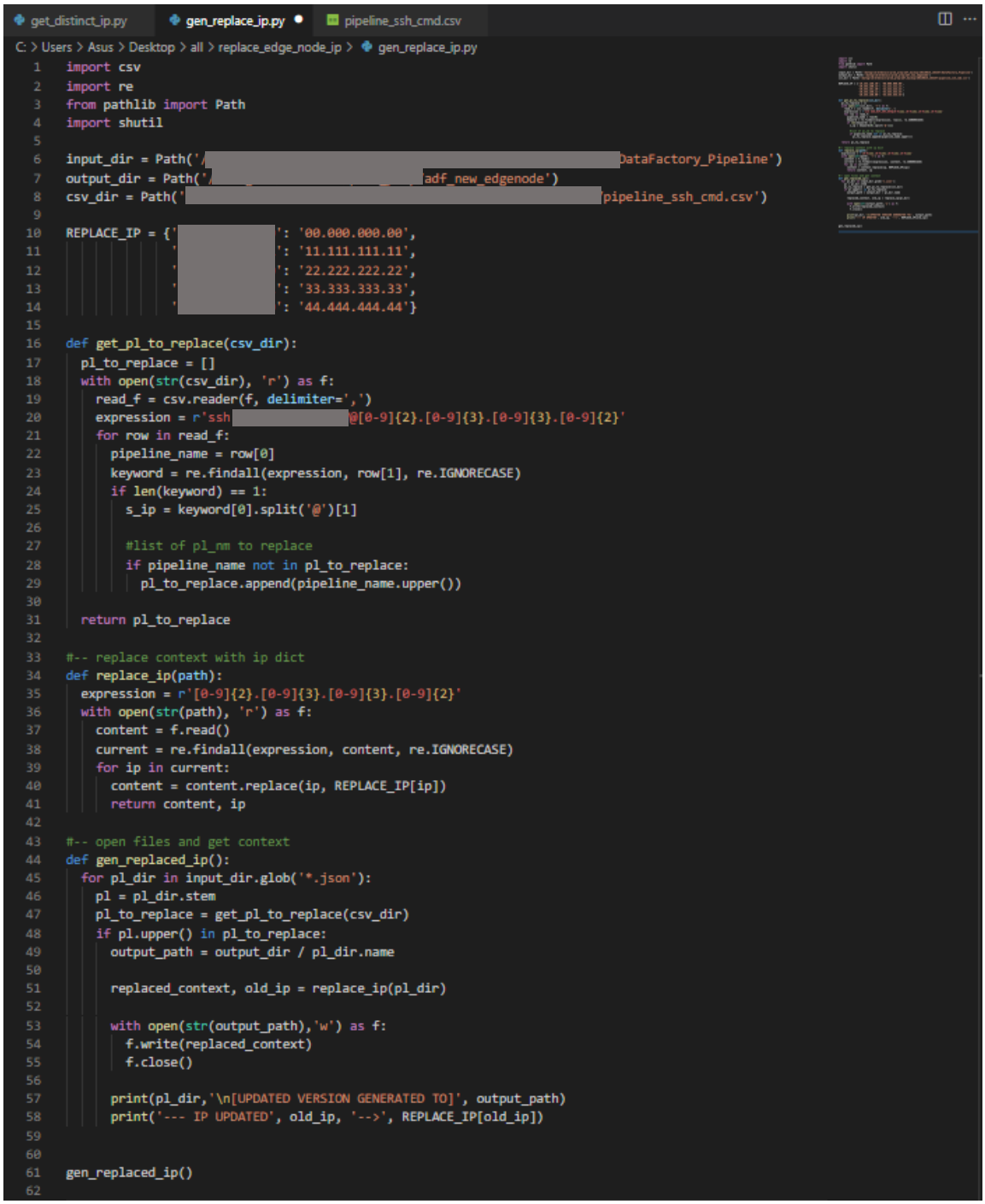
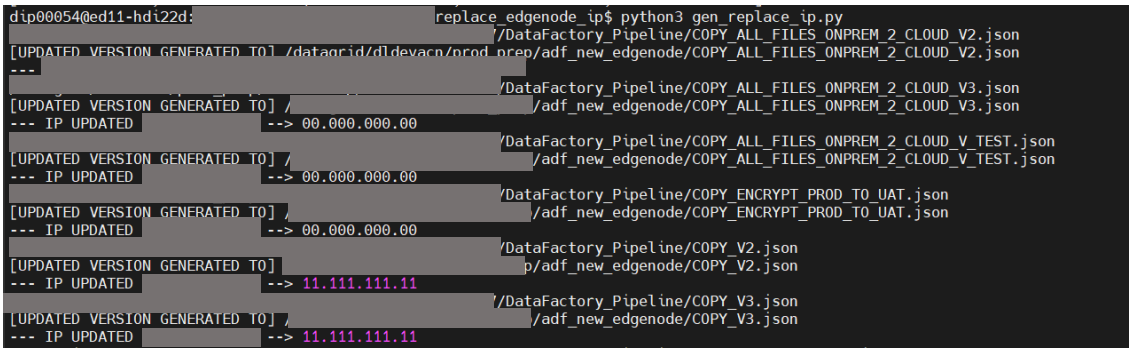


Figure 3.25 Screenshot of generateing files with replaced IP address code.



Screenshot of generating files with replaced IP address output.

3.3.2.4 Validate date function

The objective of this task is to test the date functions and compare the result between HDI3.6 and 4.0 to validate that both return the same result and ensures data compatibility. Each test scenario will be test with 10 different cases which consider the variations of result query from each HDI version and its datatype. (check data stored on 3.6 could read correctly on 4.0, value must be equal)

The test will be looking on two parts: 1. Query results selecting directly on hive in HDI3.6 and 4.0., 2. Query results selecting from tables across hive HDI3.6 and 4.0 and specifically focusing on the result stored as string and timestamp datatype.

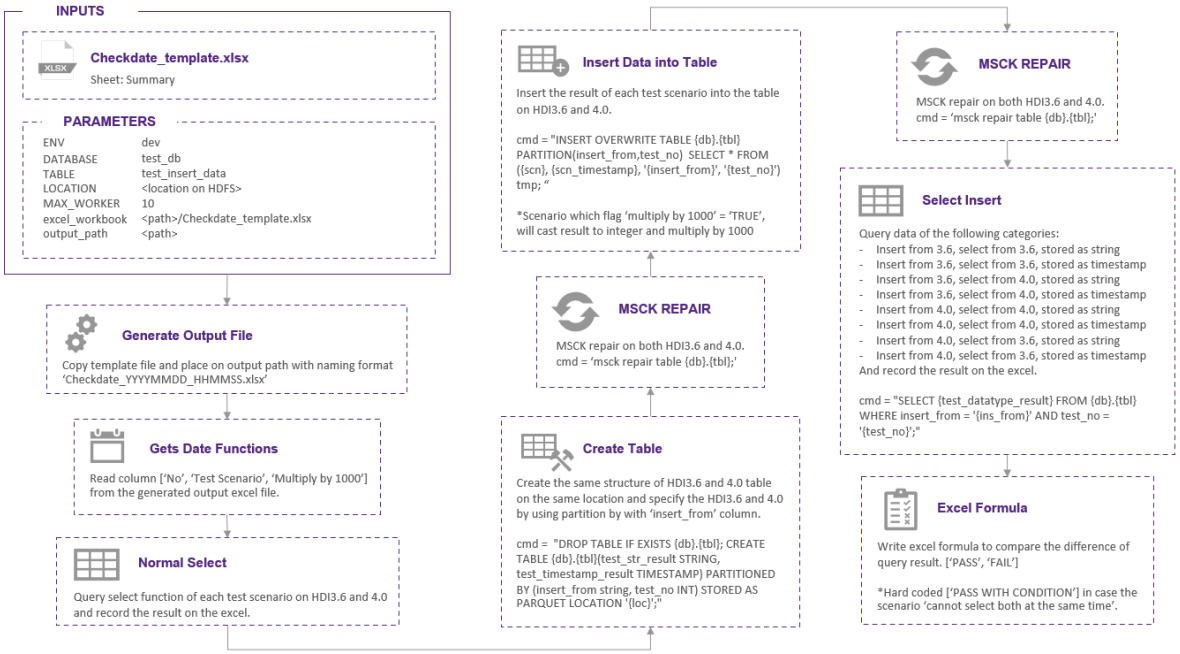


Figure 3.26 Workflow chart of validating date function.

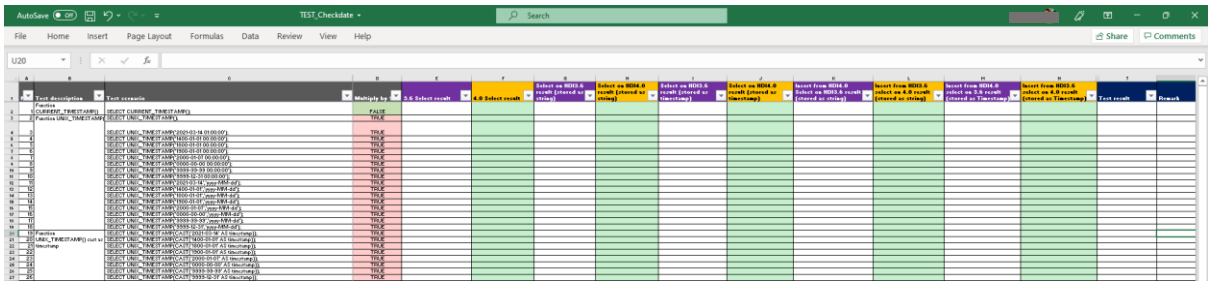
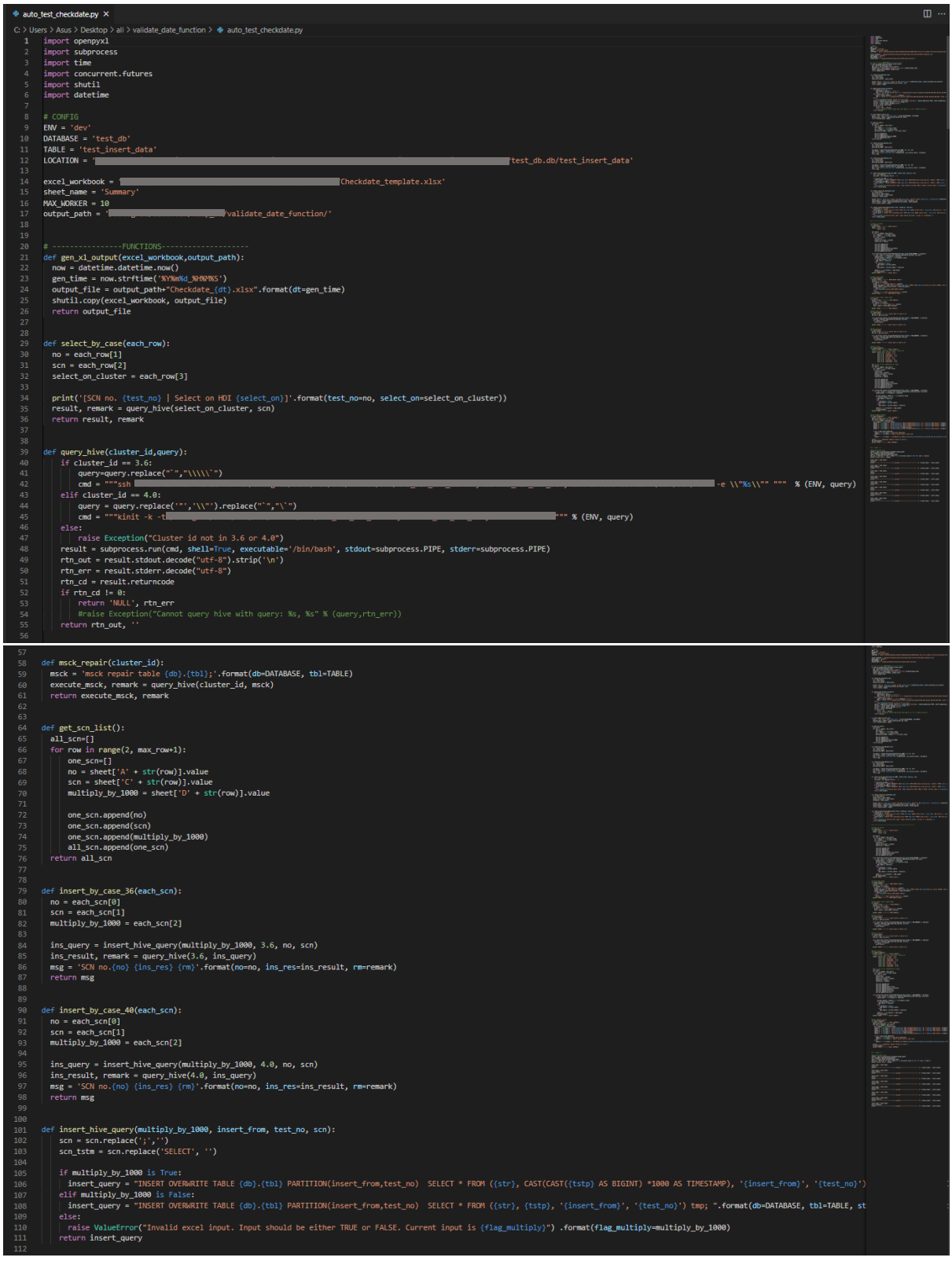


Figure 3.27 Screenshot of input excel file.



```python
import openpyxl
import subprocess
import time
import concurrent.futures
import shutil
import datetime

# CONFIG
ENV = 'dev'
DATABASE = 'test_db'
TABLE = 'test_insert_data'
LOCATION = '[redacted]test_db.db/test_insert_data'

excel_workbook = '[redacted]Checkdate_template.xlsx'
sheet_name = 'Summary'
MAX_WORKER = 10
output_path = '[redacted]/validate_date_function/'


# ----------------FUNCTIONS--------------------
def gen_xl_output(excel_workbook,output_path):
  now = datetime.datetime.now()
  gen_time = now.strftime('%Y%m%d_%H%M%S')
  output_file = output_path+"Checkdate_{dt}.xlsx".format(dt=gen_time)
  shutil.copy(excel_workbook, output_file)
  return output_file


def select_by_case(each_row):
  no = each_row[1]
  scn = each_row[2]
  select_on_cluster = each_row[3]

  print('[SCN no. {test_no} | Select on HDI {select_on}]'.format(test_no=no, select_on=select_on_cluster))
  result, remark = query_hive(select_on_cluster, scn)
  return result, remark


def query_hive(cluster_id,query):
    if cluster_id == 3.6:
        query=query.replace("`","\\\\`")
        cmd = """ssh [redacted]-e \\"%s\\"" """  % (ENV, query)
    elif cluster_id == 4.0:
        query = query.replace('"','\\"').replace("`","\`")
        cmd = """kinit -k -t[redacted]""" % (ENV, query)
    else:
        raise Exception("Cluster id not in 3.6 or 4.0")
    result = subprocess.run(cmd, shell=True, executable='/bin/bash', stdout=subprocess.PIPE, stderr=subprocess.PIPE)
    rtn_out = result.stdout.decode("utf-8").strip('\n')
    rtn_err = result.stderr.decode("utf-8")
    rtn_cd = result.returncode
    if rtn_cd != 0:
        return 'NULL', rtn_err
        #raise Exception("Cannot query hive with query: %s, %s" % (query,rtn_err))
    return rtn_out, ''


def msck_repair(cluster_id):
  msck = 'msck repair table {db}.{tbl};'.format(db=DATABASE, tbl=TABLE)
  execute_msck, remark = query_hive(cluster_id, msck)
  return execute_msck, remark


def get_scn_list():
  all_scn=[]
  for row in range(2, max_row+1):
      one_scn=[]
      no = sheet['A' + str(row)].value
      scn = sheet['C' + str(row)].value
      multiply_by_1000 = sheet['D' + str(row)].value

      one_scn.append(no)
      one_scn.append(scn)
      one_scn.append(multiply_by_1000)
      all_scn.append(one_scn)
  return all_scn


def insert_by_case_36(each_scn):
  no = each_scn[0]
  scn = each_scn[1]
  multiply_by_1000 = each_scn[2]

  ins_query = insert_hive_query(multiply_by_1000, 3.6, no, scn)
  ins_result, remark = query_hive(3.6, ins_query)
  msg = 'SCN no.{no} {ins_res} {rm}'.format(no=no, ins_res=ins_result, rm=remark)
  return msg


def insert_by_case_40(each_scn):
  no = each_scn[0]
  scn = each_scn[1]
  multiply_by_1000 = each_scn[2]

  ins_query = insert_hive_query(multiply_by_1000, 4.0, no, scn)
  ins_result, remark = query_hive(4.0, ins_query)
  msg = 'SCN no.{no} {ins_res} {rm}'.format(no=no, ins_res=ins_result, rm=remark)
  return msg


def insert_hive_query(multiply_by_1000, insert_from, test_no, scn):
    scn = scn.replace(';','')
    scn_tstm = scn.replace('SELECT', '')

    if multiply_by_1000 is True:
      insert_query = "INSERT OVERWRITE TABLE {db}.{tbl} PARTITION(insert_from,test_no)  SELECT * FROM ({str}, CAST(CAST({tstp} AS BIGINT) *1000 AS TIMESTAMP), '{insert_from}', '{test_no}')
    elif multiply_by_1000 is False:
      insert_query = "INSERT OVERWRITE TABLE {db}.{tbl} PARTITION(insert_from,test_no)  SELECT * FROM ({str}, {tstp}, '{insert_from}', '{test_no}') tmp; ".format(db=DATABASE, tbl=TABLE, st
    else:
      raise ValueError("Invalid excel input. Input should be either TRUE or FALSE. Current input is {flag_multiply}") .format(flag_multiply=multiply_by_1000)
    return insert_query
```

Figure 3.28 Screenshot of automate test date function code.



```
def select_insert_by_case(each_row):
  no = each_row[1]
  insert_from = each_row[2]
  select_on_cluster = each_row[3]
  stored_as = each_row[4]

  print('[SCN no. {test_no} | Insert from HDI {ins_from} | Select on HDI {select_on} | {stored_as}]'.format(test_no=no, ins_from=insert_from, select_on=select_on_cluster, stored_as=stored_
  select_query = insert_select_hive_query(insert_from, stored_as, no)
  select_result, remark = query_hive(select_on_cluster, select_query)
  return select_result, remark


def insert_select_hive_query(insert_from, stored_as, test_no):
  if stored_as == 'string':
    select_query = "SELECT test_str_result FROM {db}.{tbl} WHERE insert_from = '{ins_from}' AND test_no = '{test_no}';".format(db=DATABASE, tbl=TABLE, ins_from=insert_from, test_no=test_no
  elif stored_as == 'timestamp':
    select_query = "SELECT test_timestamp_result FROM {db}.{tbl} WHERE insert_from = '{ins_from}' AND test_no = '{test_no}';".format(db=DATABASE, tbl=TABLE, ins_from=insert_from, test_no=t
  else:
    raise ValueError("Invalid excel input. Input should be either 'string' or 'timestamp'.")
  return select_query
```

```
#-------------------------------------------------------------


# select no insert
def select():
  print('[DONE] --------- SELECT date')
  cases = [[3.6, 'E'],
           [4.0, 'F']]

  all_row=[]
  for row in range(2, max_row+1):
    no = sheet['A' + str(row)].value
    scn = sheet['C' + str(row)].value
    for case in cases:
      row_list=[]
      select_on_cluster = case[0]
      result_col = case[1]

      row_list.append(row)
      row_list.append(no)
      row_list.append(scn)
      row_list.append(select_on_cluster)
      row_list.append(result_col)
      all_row.append(row_list)

  with concurrent.futures.ProcessPoolExecutor(max_workers=MAX_WORKER) as executor:
    for row, result in zip(all_row, executor.map(select_by_case, all_row)):
      sheet[row[4] + str(row[0])] = result[0]
      current_remark = sheet['U' + str(row[0])].value
      if current_remark is None:
        add_remark = result[1]
      else:
        if result[1] is None:
          add_remark = current_remark
        else:
          add_remark = current_remark + result[1]

      sheet['U' + str(row[0])] = add_remark
      wb.save(output)
  print('[DONE] --------- SELECT date')


# drop/create table
def drop_create():
  print('[START] --------- DROP/CREATE TABLE')
  clusters = [3.6, 4.0]
  for cluster in clusters:
    print('------- creating table on', cluster)
    create_tbl_query = "DROP TABLE IF EXISTS {db}.{tbl}; CREATE TABLE {db}.{tbl}(test_str_result STRING, test_timestamp_result TIMESTAMP) PARTITIONED BY (insert_from string, test_no INT) S
    table, remark = query_hive(cluster, create_tbl_query)
    if table == 'NULL':
      raise Exception('Cannot DROP/CREATE TABLE')
    else:
      print('------- done creating table on', cluster)
  print('[DONE] --------- DROP/CREATE TABLE')
```

```
# msck repair after create table
def msck():
  print('[START] ---------- MSCK REPAIR')
  clusters = [3.6, 4.0]
  for cluster in clusters:
    print('-------- msck repair on', cluster)
    msck, remark = msck_repair(cluster)

  print('[DONE] ---------- MSCK REPAIR')

#insert into 3.6
def insert_36():
  print('[START] --------- Insert date to table 3.6')
  all_scn = get_scn_list()

  with concurrent.futures.ProcessPoolExecutor(max_workers = MAX_WORKER) as executor:
    results = executor.map(insert_by_case_36, all_scn)
    for result in results:
      print(result)

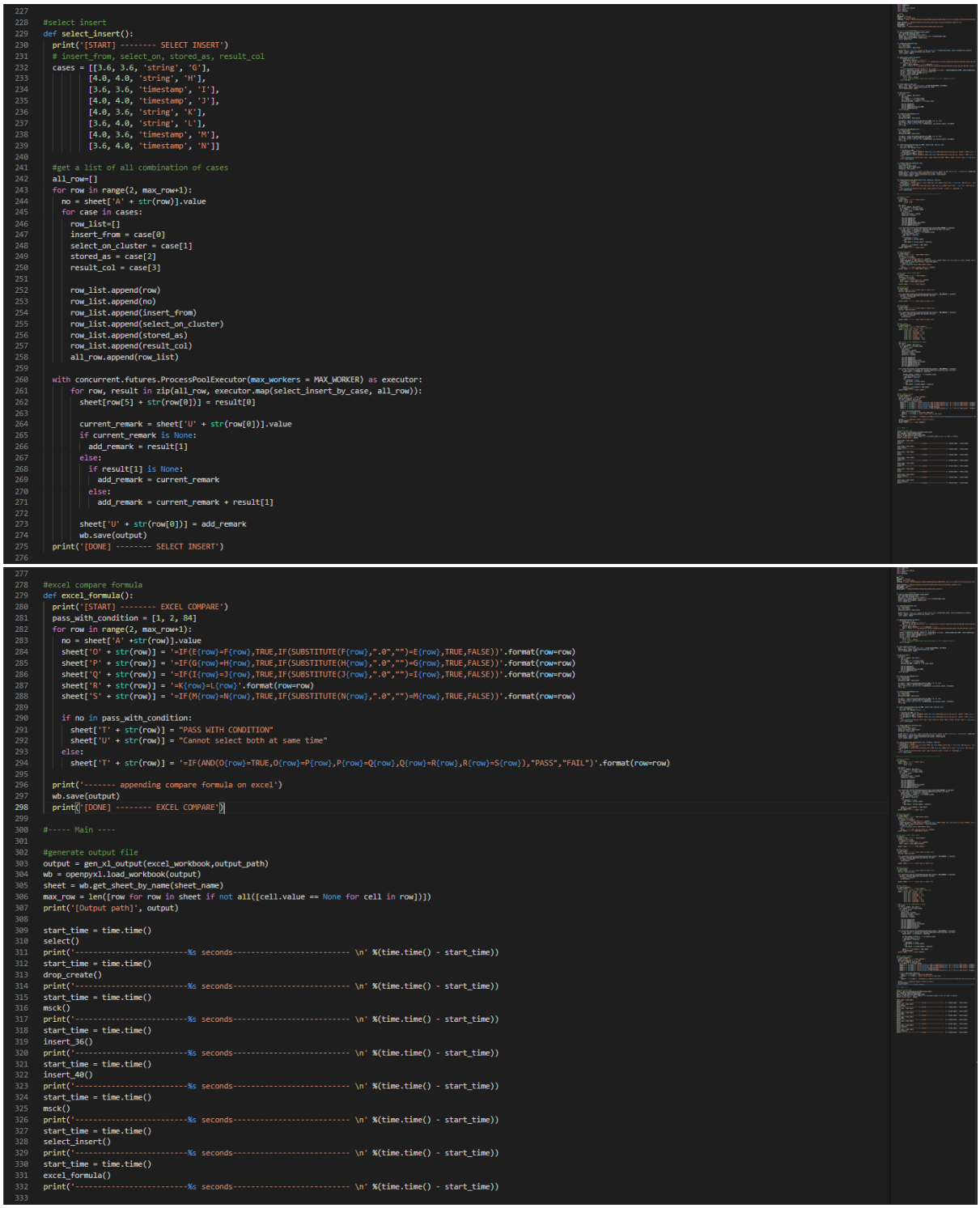
  print('[DONE] --------- Insert date to table 3.6')


#insert into 4.0
def insert_40():
  print('[START] --------- Insert date to table 4.0')
  all_scn = get_scn_list()

  with concurrent.futures.ProcessPoolExecutor(max_workers = MAX_WORKER) as executor:
    results = executor.map(insert_by_case_40, all_scn)
    for result in results:
      print(result)

  print('[DONE] --------- Insert date to table 4.0')
```

Figure 3.29 Screenshot of automate test date function code (cont.)



```python
#select insert
def select_insert():
  print('[START] -------- SELECT INSERT')
  # insert_from, select_on, stored_as, result_col
  cases = [[3.6, 3.6, 'string', 'G'],
           [4.0, 4.0, 'string', 'H'],
           [3.6, 3.6, 'timestamp', 'I'],
           [4.0, 4.0, 'timestamp', 'J'],
           [4.0, 3.6, 'string', 'K'],
           [3.6, 4.0, 'string', 'L'],
           [4.0, 3.6, 'timestamp', 'M'],
           [3.6, 4.0, 'timestamp', 'N']]

  #get a list of all combination of cases
  all_row=[]
  for row in range(2, max_row+1):
    no = sheet['A' + str(row)].value
    for case in cases:
      row_list=[]
      insert_from = case[0]
      select_on_cluster = case[1]
      stored_as = case[2]
      result_col = case[3]

      row_list.append(row)
      row_list.append(no)
      row_list.append(insert_from)
      row_list.append(select_on_cluster)
      row_list.append(stored_as)
      row_list.append(result_col)
      all_row.append(row_list)

  with concurrent.futures.ProcessPoolExecutor(max_workers = MAX_WORKER) as executor:
      for row, result in zip(all_row, executor.map(select_insert_by_case, all_row)):
        sheet[row[5] + str(row[0])] = result[0]

        current_remark = sheet['U' + str(row[0])].value
        if current_remark is None:
          add_remark = result[1]
        else:
          if result[1] is None:
            add_remark = current_remark
          else:
            add_remark = current_remark + result[1]

        sheet['U' + str(row[0])] = add_remark
        wb.save(output)
  print('[DONE] -------- SELECT INSERT')
```

```python
#excel compare formula
def excel_formula():
  print('[START] -------- EXCEL COMPARE')
  pass_with_condition = [1, 2, 84]
  for row in range(2, max_row+1):
    no = sheet['A' +str(row)].value
    sheet['O' + str(row)] = '=IF(E{row}=F{row},TRUE,IF(SUBSTITUTE(F{row},".0","")=E{row},TRUE,FALSE))'.format(row=row)
    sheet['P' + str(row)] = '=IF(G{row}=H{row},TRUE,IF(SUBSTITUTE(H{row},".0","")=G{row},TRUE,FALSE))'.format(row=row)
    sheet['Q' + str(row)] = '=IF(I{row}=J{row},TRUE,IF(SUBSTITUTE(J{row},".0","")=I{row},TRUE,FALSE))'.format(row=row)
    sheet['R' + str(row)] = '=K{row}=L{row}'.format(row=row)
    sheet['S' + str(row)] = '=IF(M{row}=N{row},TRUE,IF(SUBSTITUTE(N{row},".0","")=M{row},TRUE,FALSE))'.format(row=row)

    if no in pass_with_condition:
      sheet['T' + str(row)] = "PASS WITH CONDITION"
      sheet['U' + str(row)] = "Cannot select both at same time"
    else:
      sheet['T' + str(row)] = '=IF(AND(O{row}=TRUE,O{row}=P{row},P{row}=Q{row},Q{row}=R{row},R{row}=S{row}),"PASS","FAIL")'.format(row=row)

  print('------- appending compare formula on excel')
  wb.save(output)
  print('[DONE] -------- EXCEL COMPARE')

#----- Main ----

#generate output file
output = gen_xl_output(excel_workbook,output_path)
wb = openpyxl.load_workbook(output)
sheet = wb.get_sheet_by_name(sheet_name)
max_row = len([row for row in sheet if not all([cell.value == None for cell in row])])
print('[Output path]', output)

start_time = time.time()
select()
print('-------------------------%s seconds------------------------- \n' %(time.time() - start_time))
start_time = time.time()
drop_create()
print('-------------------------%s seconds------------------------- \n' %(time.time() - start_time))
start_time = time.time()
msck()
print('-------------------------%s seconds------------------------- \n' %(time.time() - start_time))
start_time = time.time()
insert_36()
print('-------------------------%s seconds------------------------- \n' %(time.time() - start_time))
start_time = time.time()
insert_40()
print('-------------------------%s seconds------------------------- \n' %(time.time() - start_time))
start_time = time.time()
msck()
print('-------------------------%s seconds------------------------- \n' %(time.time() - start_time))
start_time = time.time()
select_insert()
print('-------------------------%s seconds------------------------- \n' %(time.time() - start_time))
start_time = time.time()
excel_formula()
print('-------------------------%s seconds------------------------- \n' %(time.time() - start_time))
```

Figure 3.30 Screenshot of automate test date function code (cont.)

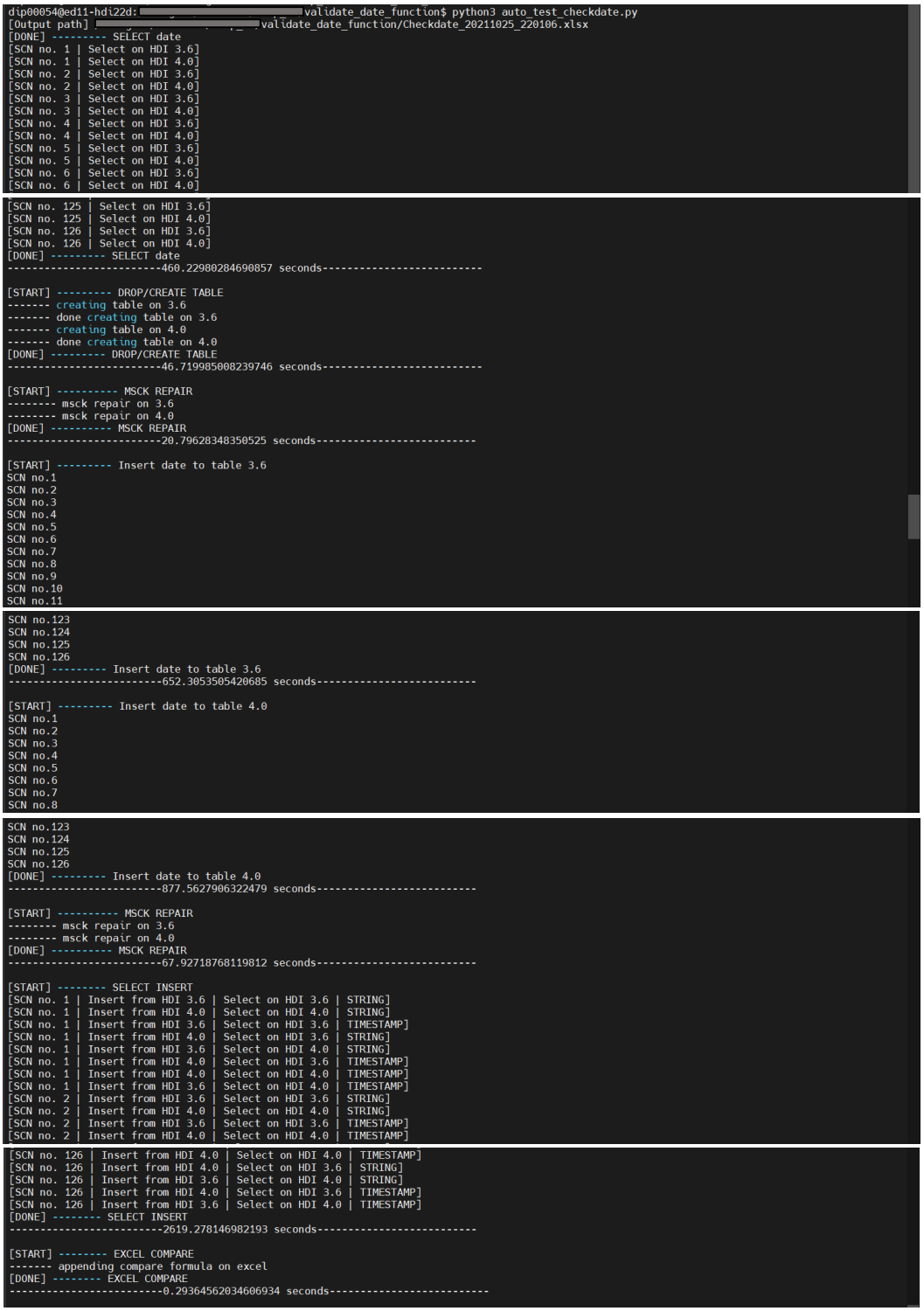


Figure 3.31 Screenshot of automate test date function output.

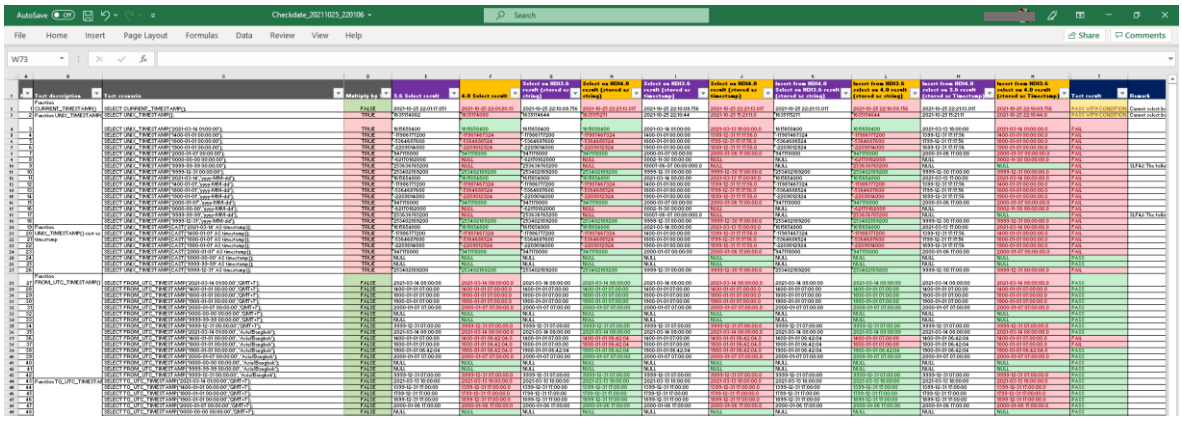


Figure 3.32 Screenshot of auto test date function output excel file.

3.3.2.5 Validate HDI4.0 insert overwrite with empty result

This is the process of validating the hive bug fix. Wherein, hive cannot insert overwrite data with empty result on HDI4.0. This task is used to test and ensure that it is successfully working after the bug was fixed.

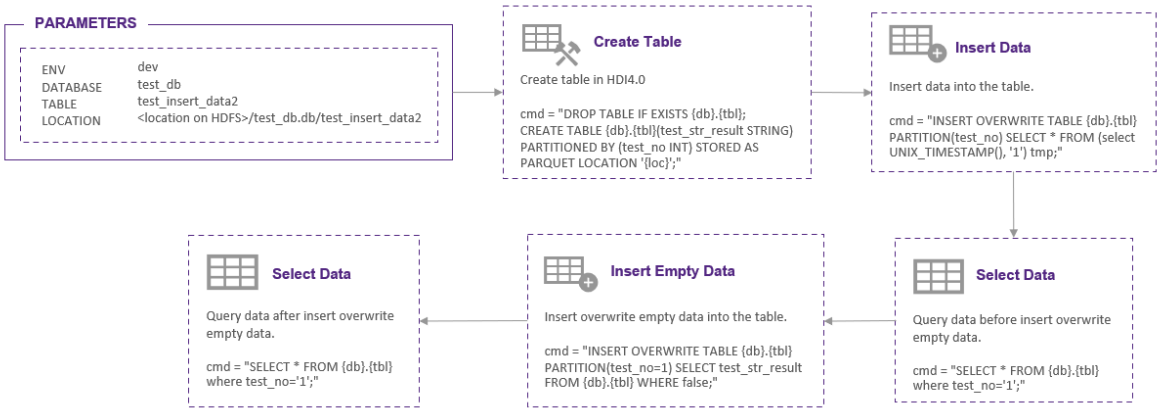
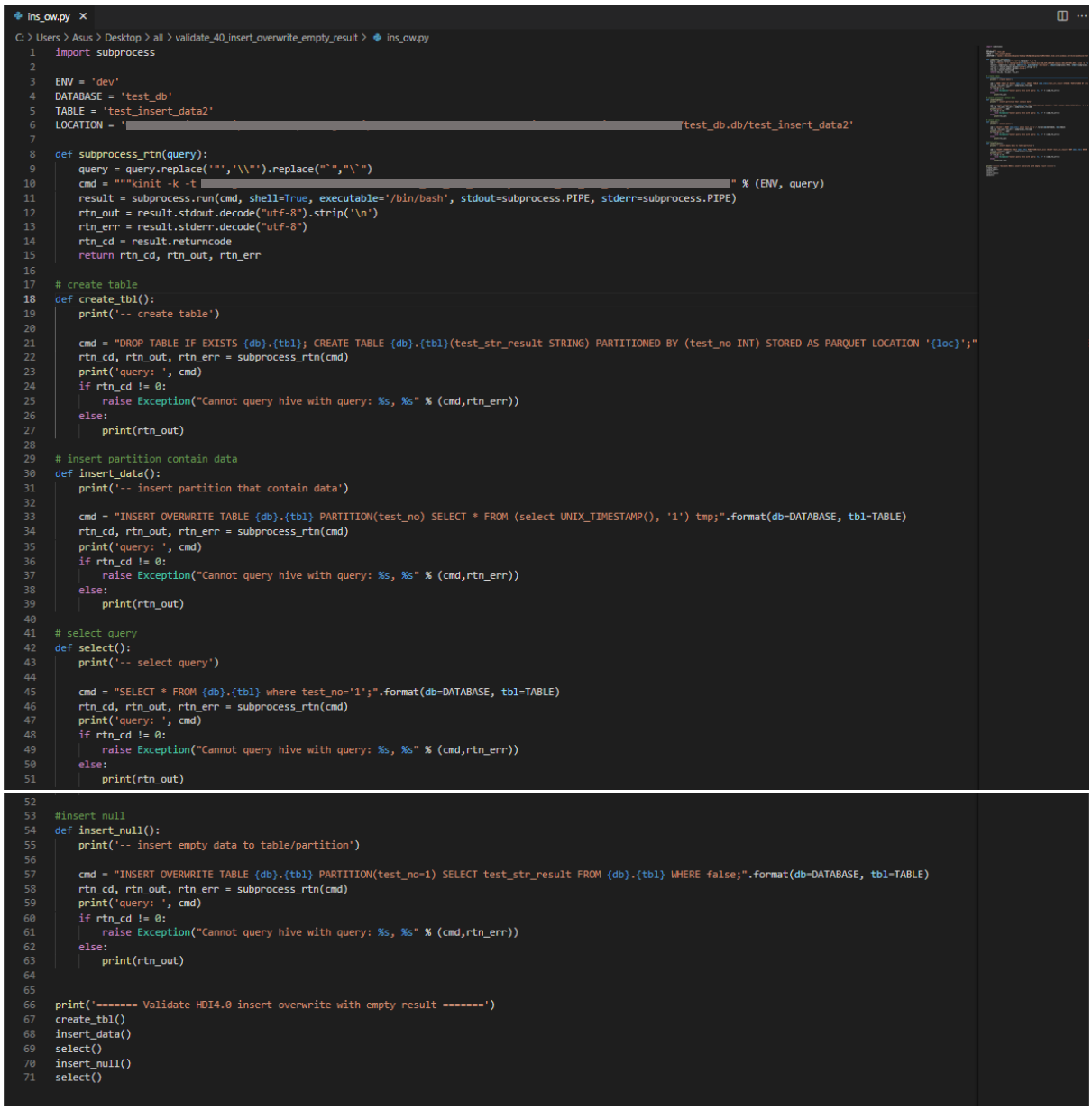


Figure 3.33 Workflow chart of validating insert overwrite with empty result.



```
ins_ow.py
C: > Users > Asus > Desktop > all > validate_40_insert_overwrite_empty_result > ins_ow.py
import subprocess

ENV = 'dev'
DATABASE = 'test_db'
TABLE = 'test_insert_data2'
LOCATION = '                                                
```

Figure 3.35 Screenshot of validating insert overwrite with empty result output.

3.3.2.6 Validate HDI4.0 drop partition with varchar datatype

This is the process of validating the hive bug fix. Wherein, hive cannot drop partition with varchar datatype on HDI4.0. This task is used to test and ensure that it is successfully working after the bug was fixed.

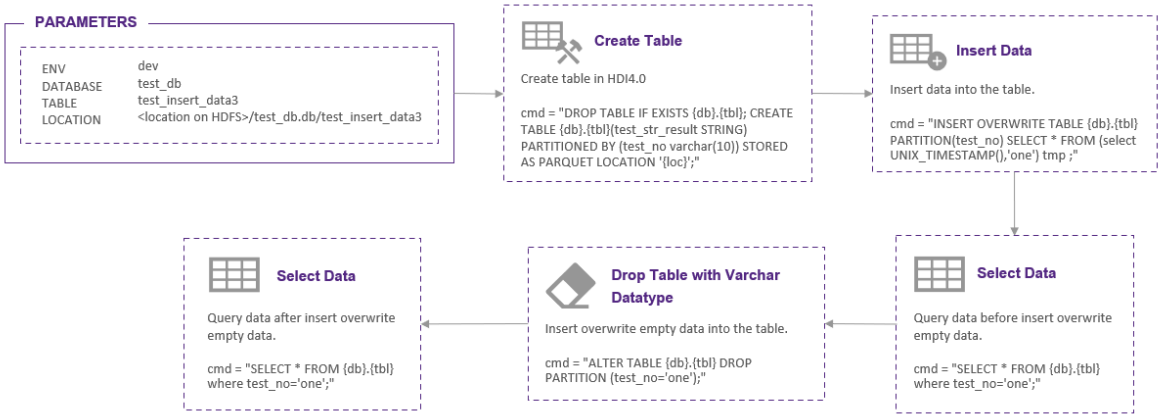
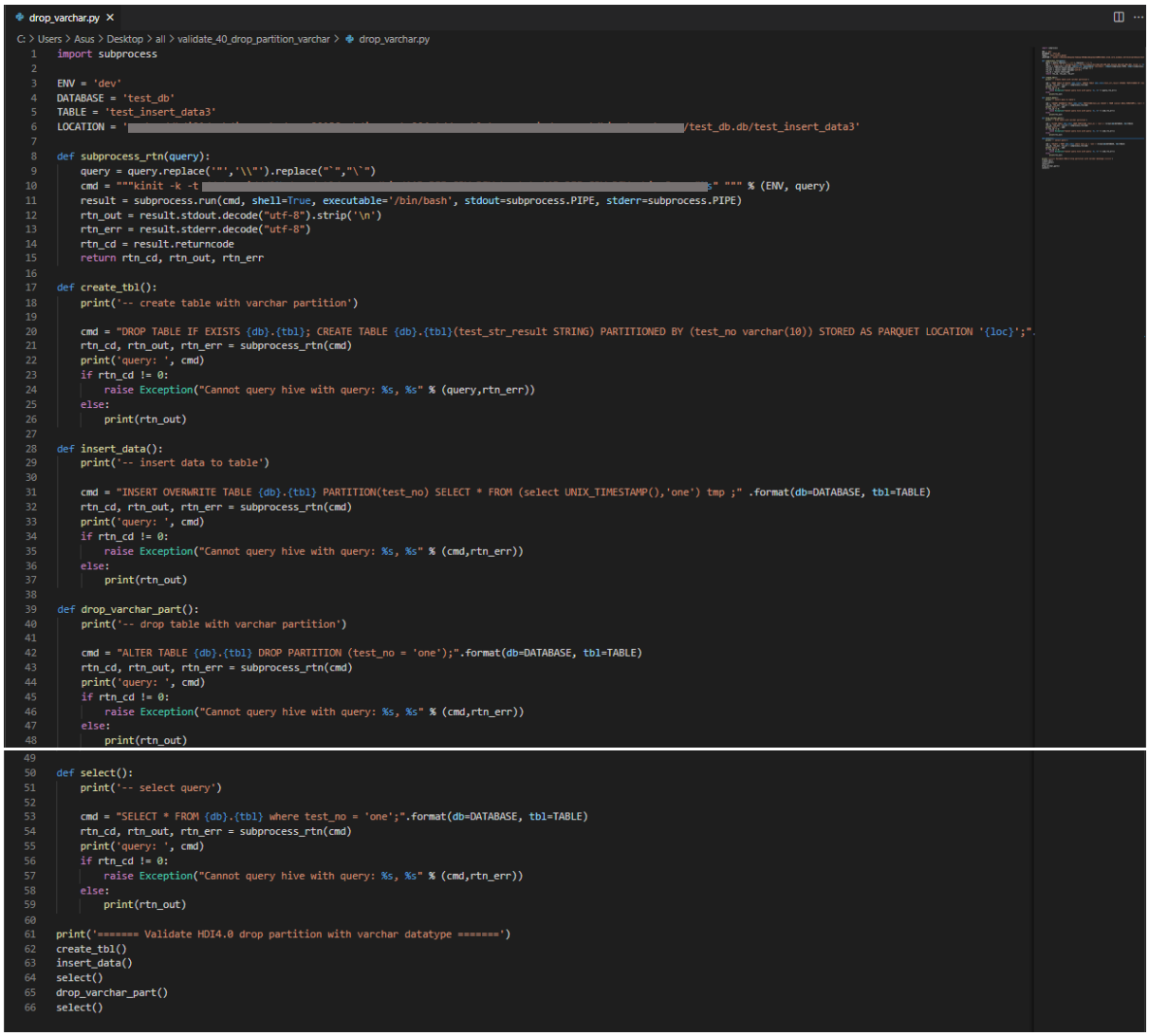


Figure 3.36 Workflow chart of validating drop partition with varchar datatype.



```python
import subprocess

ENV = 'dev'
DATABASE = 'test_db'
TABLE = 'test_insert_data3'
LOCATION = '[redacted]/test_db.db/test_insert_data3'

def subprocess_rtn(query):
    query = query.replace('"','\\"').replace("`","\`")
    cmd = """kinit -k -t [redacted]" """ % (ENV, query)
    result = subprocess.run(cmd, shell=True, executable='/bin/bash', stdout=subprocess.PIPE, stderr=subprocess.PIPE)
    rtn_out = result.stdout.decode("utf-8").strip('\n')
    rtn_err = result.stderr.decode("utf-8")
    rtn_cd = result.returncode
    return rtn_cd, rtn_out, rtn_err

def create_tbl():
    print('-- create table with varchar partition')

    cmd = "DROP TABLE IF EXISTS {db}.{tbl}; CREATE TABLE {db}.{tbl}(test_str_result STRING) PARTITIONED BY (test_no varchar(10)) STORED AS PARQUET LOCATION '{loc}';".
    rtn_cd, rtn_out, rtn_err = subprocess_rtn(cmd)
    print('query: ', cmd)
    if rtn_cd != 0:
        raise Exception("Cannot query hive with query: %s, %s" % (query,rtn_err))
    else:
        print(rtn_out)

def insert_data():
    print('-- insert data to table')

    cmd = "INSERT OVERWRITE TABLE {db}.{tbl} PARTITION(test_no) SELECT * FROM (select UNIX_TIMESTAMP(),'one') tmp ;" .format(db=DATABASE, tbl=TABLE)
    rtn_cd, rtn_out, rtn_err = subprocess_rtn(cmd)
    print('query: ', cmd)
    if rtn_cd != 0:
        raise Exception("Cannot query hive with query: %s, %s" % (cmd,rtn_err))
    else:
        print(rtn_out)

def drop_varchar_part():
    print('-- drop table with varchar partition')

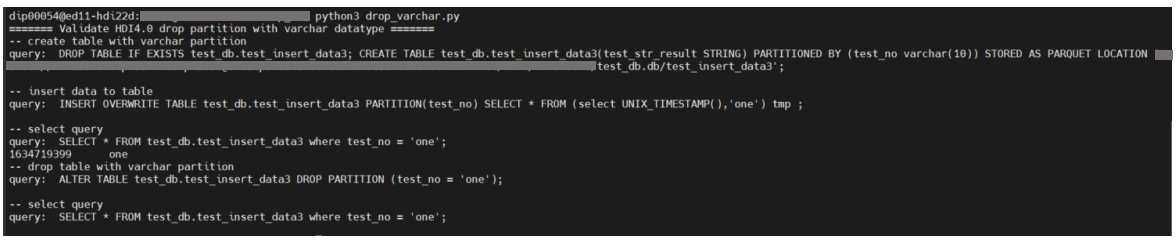
    cmd = "ALTER TABLE {db}.{tbl} DROP PARTITION (test_no = 'one');".format(db=DATABASE, tbl=TABLE)
    rtn_cd, rtn_out, rtn_err = subprocess_rtn(cmd)
    print('query: ', cmd)
    if rtn_cd != 0:
        raise Exception("Cannot query hive with query: %s, %s" % (cmd,rtn_err))
    else:
        print(rtn_out)

def select():
    print('-- select query')

    cmd = "SELECT * FROM {db}.{tbl} where test_no = 'one';".format(db=DATABASE, tbl=TABLE)
    rtn_cd, rtn_out, rtn_err = subprocess_rtn(cmd)
    print('query: ', cmd)
    if rtn_cd != 0:
        raise Exception("Cannot query hive with query: %s, %s" % (cmd,rtn_err))
    else:
        print(rtn_out)

print('======= Validate HDI4.0 drop partition with varchar datatype =======')
create_tbl()
insert_data()
select()
drop_varchar_part()
select()
```

Figure 3.37 Screenshot of validating drop partition with varchar datatype code.



```
dip00054@ed11-hdi22d:[redacted] python3 drop_varchar.py
======= Validate HDI4.0 drop partition with varchar datatype =======
-- create table with varchar partition
query:  DROP TABLE IF EXISTS test_db.test_insert_data3; CREATE TABLE test_db.test_insert_data3(test_str_result STRING) PARTITIONED BY (test_no varchar(10)) STORED AS PARQUET LOCATION [redacted]test_db.db/test_insert_data3';

-- insert data to table
query:  INSERT OVERWRITE TABLE test_db.test_insert_data3 PARTITION(test_no) SELECT * FROM (select UNIX_TIMESTAMP(),'one') tmp ;

-- select query
query:  SELECT * FROM test_db.test_insert_data3 where test_no = 'one';
1634719399      one
-- drop table with varchar partition
query:  ALTER TABLE test_db.test_insert_data3 DROP PARTITION (test_no = 'one');

-- select query
query:  SELECT * FROM test_db.test_insert_data3 where test_no = 'one';
```

Figure 3.38 Screenshot of validating drop partition with varchar datatype output.

3.3.2.7 Work transition

To ensure that the developers that will take over the work will be able to maintain the codes that have been implemented above, the transition session is scheduled. The process of this transitions is:

1. Documentation preparation of the work, which is the execution instructions including input parameters and output result of the code.
2. Transition presentation, including the explanation of the usage, workflow, execution instructions and other important point to notice. Clarify queries and receive feedback to improve the work.
3. Adjust and modify the codes and the document according to the feedback.
4. Final transition presentation, updating the points that is modified.

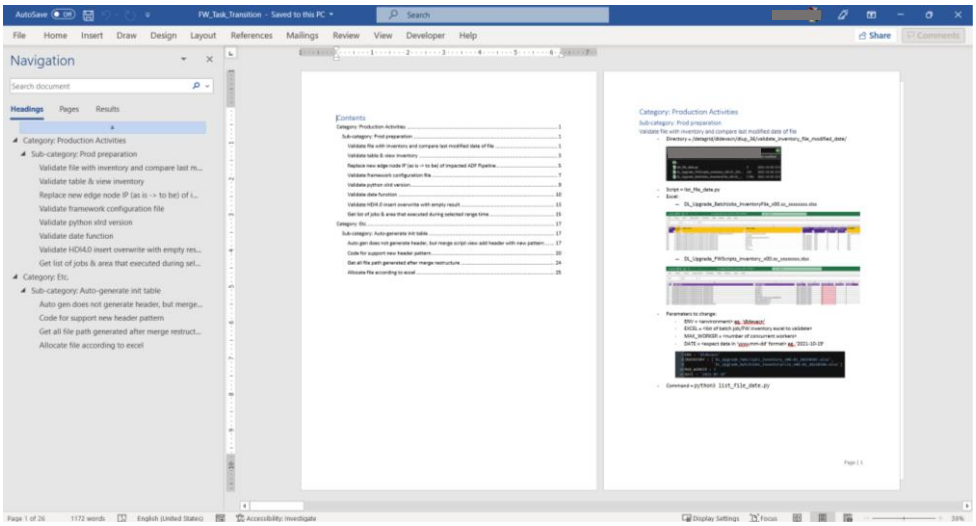


Figure 3.39 Transition document preparation.

Chapter 4

Result of the Project

4.1 Result of the Work

The result of the works done in this project will be separate into two parts which forementioned in the previous chapter.

4.1.1 Result of Jenkins Deployment Pipeline

After understanding how the Jenkins deployment pipeline work, the tools used in execution and the test scenarios, I started the unit testing and report bugs to the developers. Then, we did the testing following the test scenarios expectations and collect results as a report to the client.

Preparing documents and presentation slides for transition session. We started to plan the presenting outline and assigned each part to do. In the sessions I was the observer and write down any feedback from the session.

4.1.2 Result of Production Preparation

Implementation of Python code for the production preparation task was able to function as following (the output figures could be found in the Chapter 3 under the topic 3.3.2 onwards):

Validate file with inventory and compare last modified date of file

The program was able to show the list of files that is found and not found according to the inventory list and compare its last modified with the expectation date.

Validate table and view inventory

The program was able to show the list of tables and views that is found and not found according to the inventory list.

Replace new edge node IP (as is -> to be) of impacted ADF Pipeline

The program was able to find the distinct IP address and could generate an output file with the replaced IP version according to the IP address dictionary.

Validate date function

The program was able to automate the steps in validating the date function both in normal select and select insert record stored as string and timestamp from HDI3.6 and 4.0 and generate compare results on excel spreadsheet.

Validate HDI4.0 insert overwrite with empty result

The program was able to automate the steps in validating the bugfix in case of inserting overwrite with empty result.

Validate HDI4.0 drop partition with varchar datatype

The program was able to automate the steps in validating the bugfix in case of drop partition with varchar datatype.

4.2 Data Analysis

4.2.1 Data Analysis of Jenkins Deployment Pipeline

There is no data analysis of testing part since there were no criteria to evaluate the result. However, I was able to learn and understand the process of unit testing, software testing and was introduced to many tools such as Jenkins and SourceTree.

For the transition part, I was able to help prepare a presentation slide and document, join the transitioning session which is a very good experience to observe the presentation and how to manage the sessions with the client.

4.2.2 Data Analysis of Production Preparation

The production preparation tasks that I received were mostly validating the presence of objects, the correctness of data, and supporting the flexibility of the program execution in a different environment. The objects to validate such as inventory list and date functions were either containing huge amount of work or took a long time to process each step. These tasks can lessen the time to execute using the concept of launching parallel task.

4.3 Analysis of Results Comparing to Objectives and Aims of the Assignment

The work done in production preparation could achieve the objective of the task. In the automating validate date function task, concurrent futures functions can be optimized for a faster performance if allocate job running on two cluster at the same time.

Chapter 5

Conclusion and Recommendations

5.1 Conclusions

Referring to co-operative education program at Accenture Solutions Co. Ltd. In a duration of 5 months from 1st June 2021 to 29 October 2021, working as a data engineer analyst in a Data Lake Upgrade project. I have done mainly two works

1. Testing Jenkins Deployment Pipeline to provide information about the quality of the pipeline and prepare transitioning of the work to the clients.
2. Implement codes to automate production preparation task to reduce the effort of the work.

5.2 Problems and Possible Solutions

The problems found during working can conclude as the following:

1. Program execution time problem. Automation program execution time took too long due to the amount of input or work. To solve this, use concurrent.future module to run the program in concurrent manners which help reduce the execution time.
2. Python library not available problem. The desire library such as pandas is not available in the environment, to solve this problem I create the functions and use the data structure instead.

References

Data Lake, What is a data lake? [Online], Available : https://www.talend.com/resources/what-is-data-lake/

HDInsights, What is Azure HDInsights? [Online], Available : https://docs.microsoft.com/en-us/azure/hdinsight/hdinsight-overview

[17 June 2021]

SQL, What is SQL? [Online], Available : https://www.tutorialspoint.com/sql/sql-overview.htm

SQL, SQL PARTITION BY [Online], Available : https://www.sqltutorial.org/sql-window-functions/sql-partition-by/

Openpyxl, A Guide to Excel Spreadsheets in Python With openpyxl [Online], Available : https://realpython.com/openpyxl-excel-spreadsheets-python/

Subprocess, What is a Subprocess in Python? [Online], Available : https://geekflare.com/learn-python-subprocess/

[22 May 2021]

Concurrent.futures, Asynchronous Tasks in Python With the Concurrent.Futures Module [Online], Available : https://betterprogramming.pub/asynchronous-tasks-in-python-with-the-concurrent-futures-module-7325dc555d

[4 September 2020]

Linux Commands, Linux Commands with Examples [Online], Available :
https://www.javatpoint.com/linux-commands

Jenkins, What is Jenkins? [Online], Available : https://www.edureka.co/blog/what-is-jenkins/
[14 September 2020]

VSCode ,Visual Studio Code [Online], Available : https://en.wikipedia.org/wiki/Visual_Studio_Code
[15 October 2021]

Excel, Excel [Online], Available : https://www.computerhope.com/jargon/e/excel.htm
[11 October 2021]

Paas, SaaS vs PaaS vs IaaS: What's The Difference & How To Choose [Online], Available :
https://www.bmc.com/blogs/saas-vs-paas-vs-iaas-whats-the-difference-and-how-to-choose/#ref2
[15 June 2019]

Appendix

CCC-Co 04

ศูนย์สหกิจศึกษาและจัดหางาน สถาบันเทคโนโลยีไทย - ญี่ปุ่น

Co-operative Education and Career Center

1771/1 ถนนพัฒนาการ แขวงสวนหลวง เขตสวนหลวง กรุงเทพฯ 10250 โทรศัพท์: 0-2763-2700 ต่อ 2750, 2762 โทรสาร : 0-2763-2754

Weekly report

Week .....22.....

Name: .... Student ID: .....

Faculty: .... Major: .....

Date	Hour	Job description in summary	Knowledge and skill	Problem
Monday 25/10/21	8 hours	Transition presentation to Framework team members. Get feedback of the work.		
Tuesday 26/10/21	8 hours	Improve the work according to the feedback. Update transition document.		
Wednesday 27/10/21	8 hours	Improve the work according to the feedback. Update transition document.		
Thursday 28/10/21	8 hours	Improve the work according to the feedback. Update transition document.		
Friday 29/10/21	8 hours	Improve the work according to the feedback. Update transition document Final transition presentation of updated work.		
Saturday ...../...../.....	-			
Sunday ...../...../.....	-			
Hour of work in this report	40 hours	Certifies that this report is true and complete.	Certifies that report is true and complete.	
Hour of work in last report	773 hours	Sign..... ..... (.....) Date.....29/10/2021..... Student	Sign..... ..... (.....) Position ..... Date.....29/10/2021..... Consultant	
All hour of work	813 hours			

Note: Students must submit this report to the Cooperative Education / Internship Counselor in all faculties. Every week strictly Don't forget to copy and keep to complete the final report.

Sign Off

From the review of the cooperative education project report by supervisor has confirmed and certifies that the report is true and complete.

Signature

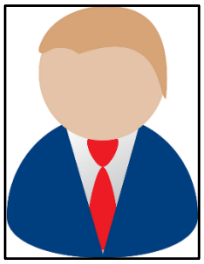
Sign _____

(xxxxxxxxxxxxxxxxxxxx)

Position: xxxxxxxxxxxxxxxxxxxx

Date _____

Background of the Writer



Full Name xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

Date of Birth xxxxxxxxxxxxxx

Educational Background

Secondary Level 2015 – 2017

Kwong Chow School (English Program)

Mathematics - Science Field

Bachelor's Degree 2018 – Present

Thai-Nichi Institute of Technology (International Program)

Faculty of Information Technology / Data Science and Analytics